



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich



Hands-on Introduction to Deep Learning with PyTorch

Swiss National Supercomputing Center (CSCS) - ETH Zurich
2-4 July 2025, Lugano



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich

Course Outline

Welcome to "Introduction to PyTorch"

July 2025



Location: Lugano



Instructors: Rafael Sarmiento, Rocco Meli, Fabian Bösch



Hosted by: CSCS / ETH Zurich

What to Expect

This workshop is hands-on and beginner-friendly. We assume:

- Familiarity with Python
- None to little exposure to basic ML concepts

Logistics & Communication

- Questions? Ask anytime!
 - We also have a Slack channel
- Course materials: <https://github.com/eth-cscs/pytorch-training>
- We'll use CSCS' ALPS cluster for the exercises
 - Login details will be provided (e-mail)

Course Overview

We'll cover:

- Fundamentals of Machine Learning
- Working with PyTorch
- Hands on exercises with JupyterLab on CSCS' ALPS cluster
- Feature extraction from text and images
- Convolutional Neural Networks (CNNs)
- Distributed training
- Introduction to Large Language Models (LLMs)

Course Material

Link to the course material and exercises can be found at: <https://github.com/eth-cscs/pytorch-training>

Timetable

	Wednesday	Thursday	Friday
Start	10:00	09:00	09:00
Morning	Introduction to Neural Networks	More on Training	Introduction to LLMs
Lunch	12:00–13:00	12:00–13:00	12:00–13:00
Afternoon	Introduction to CNNs & Feature extraction for computer vision and language modeling	Machine Room visit Distributed training	LLMs (continued)
End	17:00	17:00	16:00

Credentials for accessing the HPC systems at CSCS

- Course accounts `classxxx` will be provided to you via email
- These accounts are valid only for the duration of the course
 - Please copy any changes in the repository to your laptop before the end of the course

Course Reservations

- We have reserved a set of nodes for the course
- Access them using the Slurm reservation `pytorch`
- The Slurm account for our course is `crs01`

JupyterLab

- We will be using <https://jupyter-daint.cscs.ch>
- You can login using your credentials provided via email
- Once you logged in
 - you will find the button "Start my server" (see image)
 - Select the container image labeled as "(course)"
 - Set the job duration to 8 hours
 - Click on "Advanced options" and type "pytorch" on the "Reservation" field
 - Launch JupyterLab

The screenshot shows the JupyterLab launch interface. At the top, there are three dropdown menus: 'Queue' (set to 'Dedicated'), 'Account' (empty), and 'Job Duration' (set to '8 hours'). Below these is the 'Image' section, which contains a list of container images. The image 'nvcr.io/nvidia/pytorch:25.04-py3 (Course)' is highlighted with a red circle. To the left of this list, a red arrow points to the 'Image' label. Below the image list are two input fields: 'Custom uenv' and 'Uenv view'. Below these is the 'Reservation' section, which contains a text input field with the value 'pytorch' and a 'Custom reservation' link. At the bottom of the form is a red button labeled 'Launch JupyterLab'.

SSH Access to CSCS Cluster

- For a convenience, it is recommended to add the following lines to your

`~/.ssh/config` file

```
1 Host ela
2     HostName ela.cscs.ch
3     User <YOUR_USERNAME>
4     ForwardAgent yes
5     ForwardX11 yes
6
7 Host daint
8     HostName daint.cscs.ch
9     User <YOUR_USERNAME>
10    ProxyJump ela
11    ForwardAgent yes
12    ForwardX11 yes
```

- After that, you can connect to the cluster using the command below

```
1 $ ssh <YOUR_USERNAME>@daint
```

- You should be dropped into the `bash` shell in your home directory
- For launching jobs and we use the scratch directory

```
1 $ cd $SCRATCH
```

- here you can git-clone the course material

```
1 $ git clone https://github.com/eth-cscs/pytorch-training.git
```



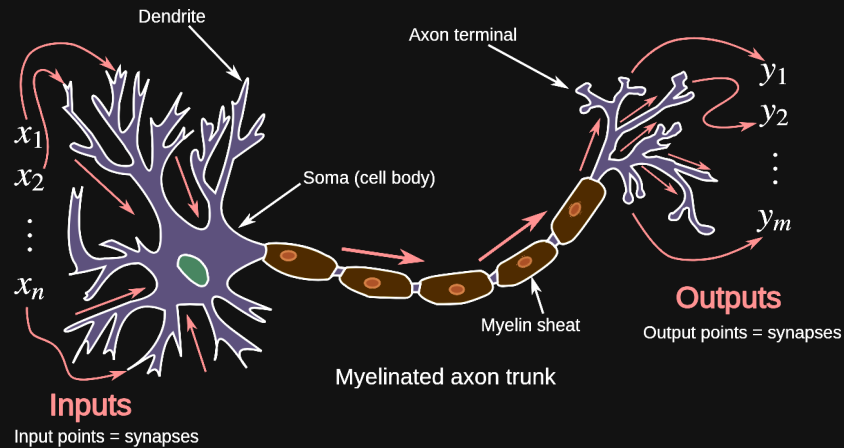
CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich

A brief introduction to Artificial Neural Networks

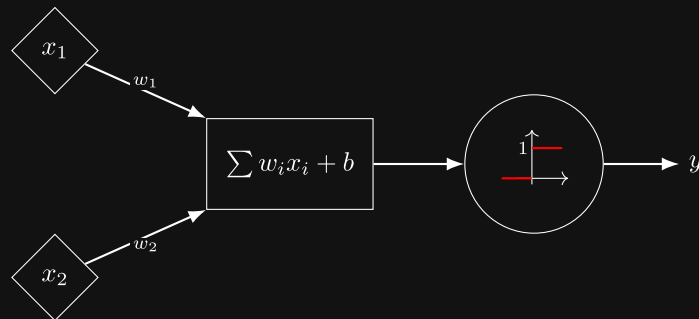
Biological Neuron



By [Egm4313.s12](#) (Prof. Loc Vu-Quoc) - Own work, [CC BY-SA 4.0](#), [Link](#)

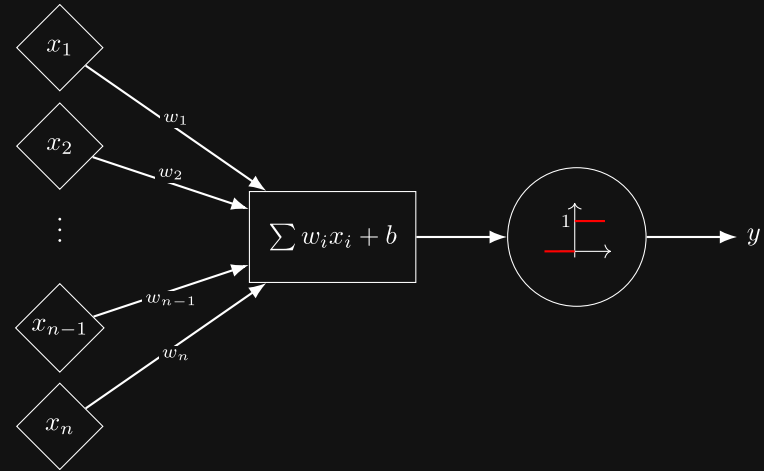
First Artificial Neuron

- Threshold Logic Unit (TLU), or Linear Threshold Unit
- Warren McCulloch and Walter Pitts in 1943
- Simple model of a biological neuron
- Fixed number of inputs and outputs
- Logical gate-like behavior (AND, OR, NOT)

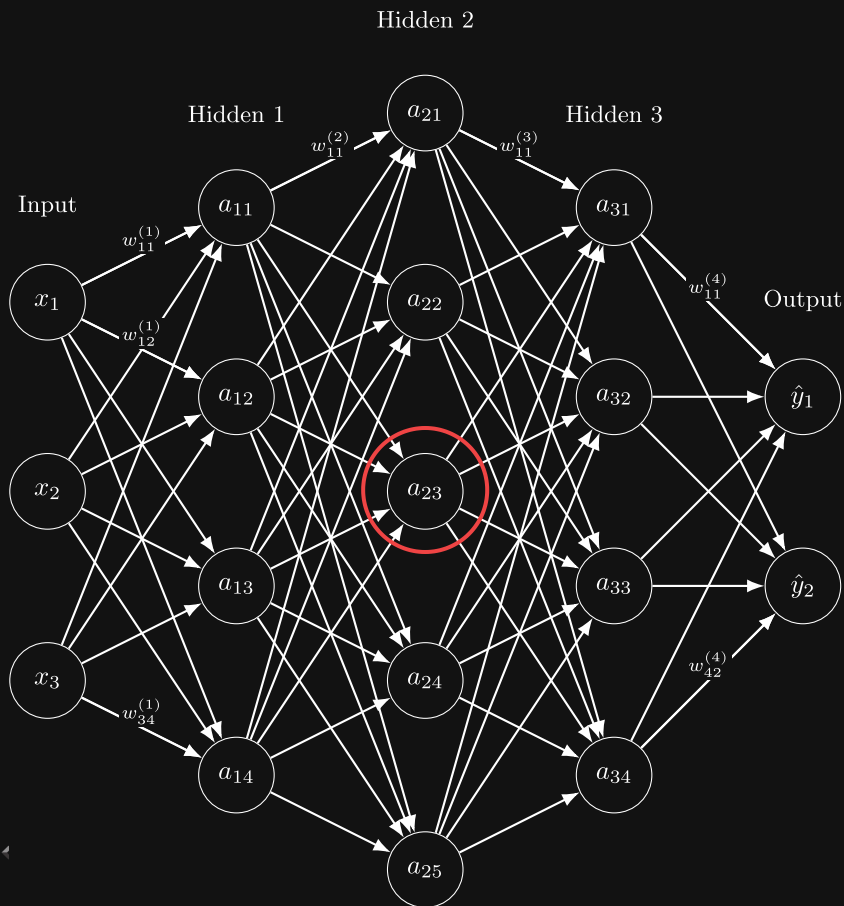


Perceptron

- Rosenblatt's Perceptron (1958)
- First algorithm for supervised learning of binary classifiers
- Maps its input features to a single output value



Multi-Layer Perceptron

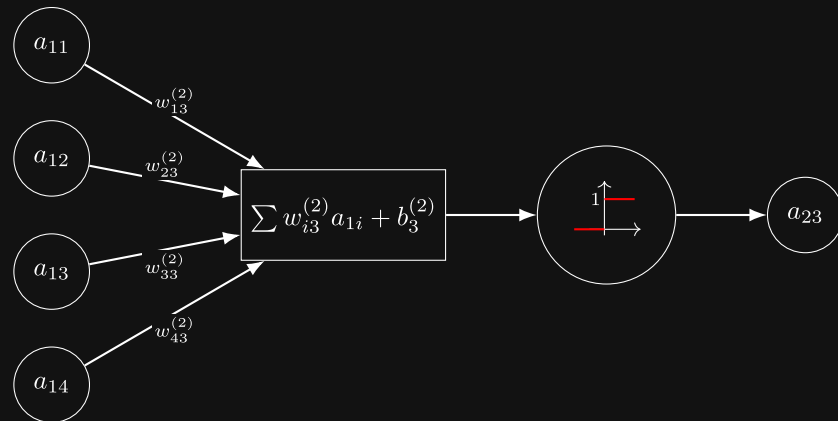


x_i : input data

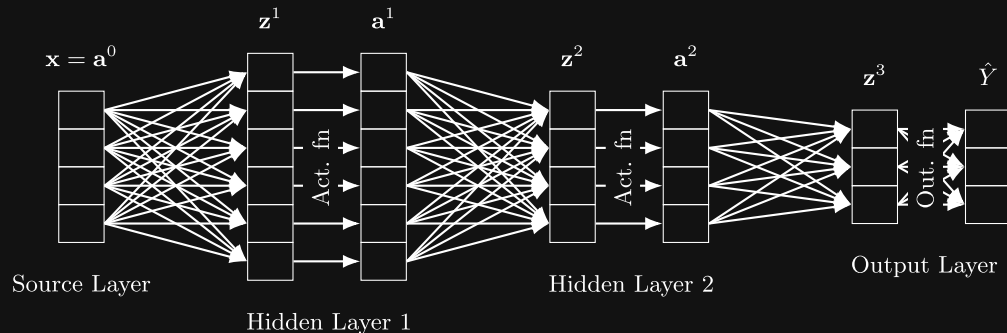
a_{ij} : activations of the j -th neuron in the i -th layer

$w_{jk}^{(i)}$: model parameters

$\hat{y}_j$: predicted output



Deep Neural Networks



- given an input $\mathbf{x}$ MLP computes the output

$$\hat{\mathbf{y}} = f^*(\mathbf{x}) \in \mathbb{R}^O$$

$$f^*(\mathbf{x}) = \dots \sigma \circ c^1(x)$$

$$f^*(\mathbf{x}) = \dots \sigma \circ c^2 \circ \sigma \circ c^1(x)$$

$$f^*(\mathbf{x}) = \sigma^O \circ c^L \circ \sigma \circ c^{L-1} \dots \sigma \circ c^2 \circ \sigma \circ c^1(x)$$

for each layer $l \in \{1, \dots, L\}$

$\mathbf{a}^{l-1} \in \mathbb{R}^{d_{l-1}}$: input

$\mathbf{z}^l \in \mathbb{R}^{d_l}$: weighted input

$\mathbf{a}^l \in \mathbb{R}^{d_l}$: output (activations)

$\mathbf{W}^l \in \mathbb{R}^{d_l \times d_{l-1}}$: weights

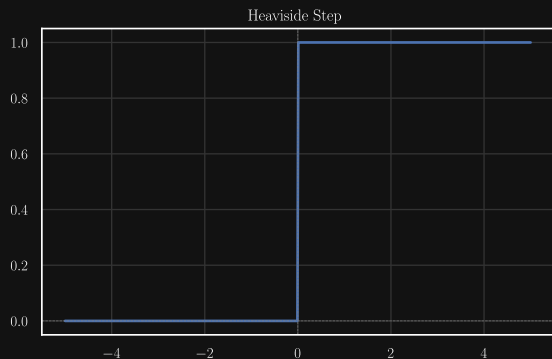
$\mathbf{b}^l \in \mathbb{R}^{d_l}$: biases

$$\mathbf{z}^l = c^l(\mathbf{a}^{l-1}) = \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l$$

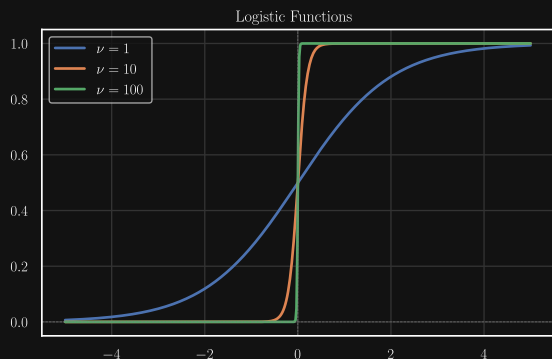
$$\mathbf{a}^l = \sigma(\mathbf{z}^l)$$

- weights $\mathbf{W} = \{\mathbf{W}^l\}_{l=1}^L$
- biases $\mathbf{B} = \{\mathbf{b}^l\}_{l=1}^L$
- parameters $\theta = \{\mathbf{W}, \mathbf{B}\}$

Activation Functions σ

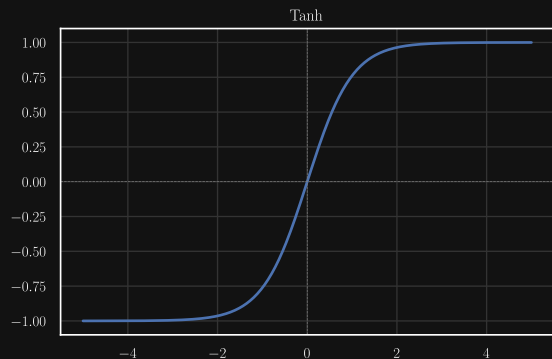


- **Heaviside step function:** $\sigma(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$
- McCulloch-Pitts neuron
- Zero gradient, bad for backpropagation
- Not used anymore

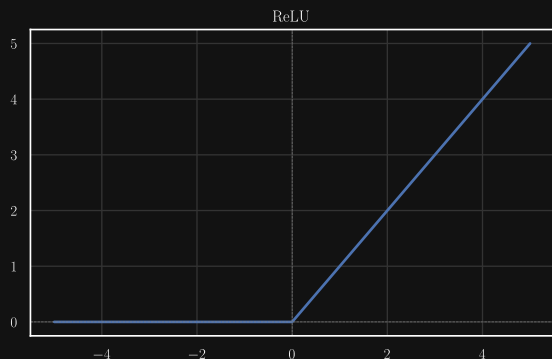


- **Logistic function:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- Smooth approximation to Heaviside function
- Good for binary classification
- Not symmetric

Activation Functions σ

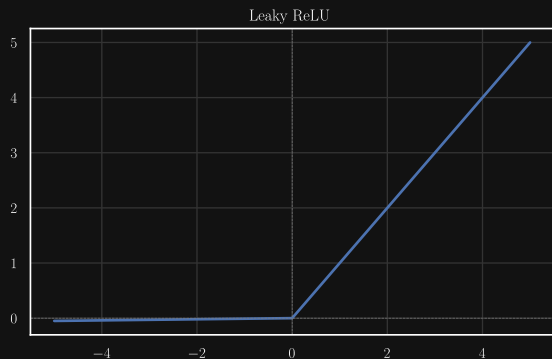


- **Tanh function:** $\sigma(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$
- Symmetric unlike Logistic function
- Smooth
- Vanishing gradients away from 0



- **ReLU (Rectified Linear Unit) function:** $\sigma(x) = \max(0, x)$
- Easy to compute
- Reduces vanishing gradient problem
- Scale invariant
- Issue of dying neurons

Activation Functions σ



- **Leaky ReLU function:** $\sigma(x) = \begin{cases} x & \text{if } x > 0 \\ \nu x & \text{if } x \leq 0 \end{cases}$, where ν is a small constant (e.g., 0.01)
- Addresses the dying ReLU problem
- Allows a small, non-zero gradient when $x < 0$
- Still retains the benefits of ReLU
- Commonly used in deep learning architectures

- many more activation functions

Supervised Learning

How can we use the data to adjust our model f parameters θ to minimize prediction error and ensure accuracy within an acceptable margin?

- Labelled data: $\{(\mathbf{y}_i, \mathbf{f}_i)\}$
- Input space: $\mathbf{y}_i \in \mathbf{Y} \subset \mathbb{R}^I$
- Output space: $\mathbf{f}_i = f(\mathbf{y}_i) \in \mathbb{R}^O$
- Training Set: $\mathcal{S} = \{(\mathbf{y}_i, \mathbf{f}_i)\}_{i=1}^N$
- Choice of training set
 - Random sampling of the input space: $\mathbf{y}_i$ i.i.d. $\sim \text{Prob}(\mathbf{Y})$

Loss Function

- Difference between predicted and actual values: $\mathbf{f}_i$ and $f_{\theta}^*(\mathbf{y}_i)$
- Evaluation: for all $\mathbf{y}_i \in \mathcal{S}$, and labels $\mathbf{f}_i \in \mathbb{R}^O$
- popular choice is the p -norm loss function
 - $p = 2$: mean squared error (MSE)
 - $p = 1$: mean absolute error (MAE), induces sparsity in the model parameters

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \underbrace{|f_{\theta}^*(\mathbf{y}_i) - \mathbf{f}_i|_p^p}_{J_i(\theta)}$$

Training (Supervised Learning)

- **Objective:** Find the model parameters θ that minimize the loss function $J(\theta)$

$$\theta^* = \arg \min_{\theta} J(\theta) = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N J_i(\theta)$$

- **Observations:**
 - $J(\theta)$ is non-convex (in general) and may have multiple local minima
 - $J(\theta)$ is differentiable with respect to θ

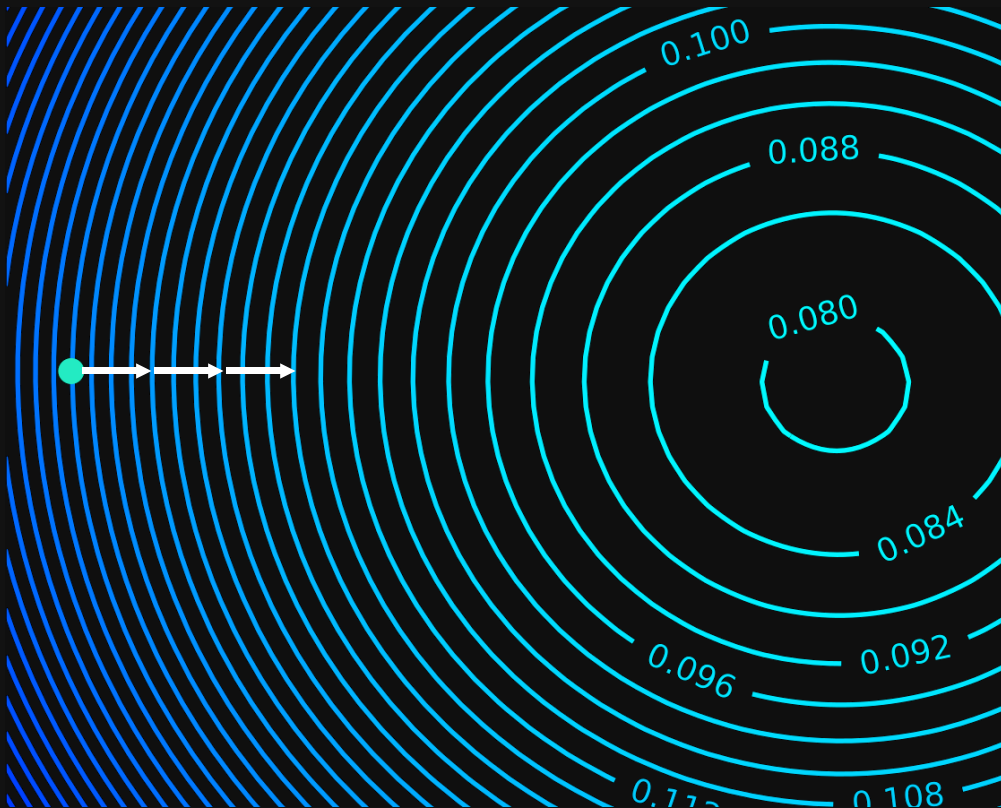
Gradient Descent

Iteratively update the model parameters in the direction of the negative gradient of the loss function

$$\theta_{l+1} = \theta_l - \eta_l \nabla_{\theta} J(\theta_l)$$

- Adaptive learning rate η_l
- Requires the computation of the gradient $\nabla_{\theta} J(\theta_l)$
- Algorithm:
 - forward pass: compute the loss $J(\theta_l)$
 - backward pass: compute the gradient $\nabla_{\theta} J(\theta_l)$
 - update the model parameters θ_{l+1} with learning rate η_l

Gradient Descent



Problems with gradient descent:

- Converges to local minima
- Slow down near saddle points
- Slow convergence due to vanishing gradients
- Divergence due to exploding gradients
- Large computational cost
 - Computation of gradients
 - Requires large memory

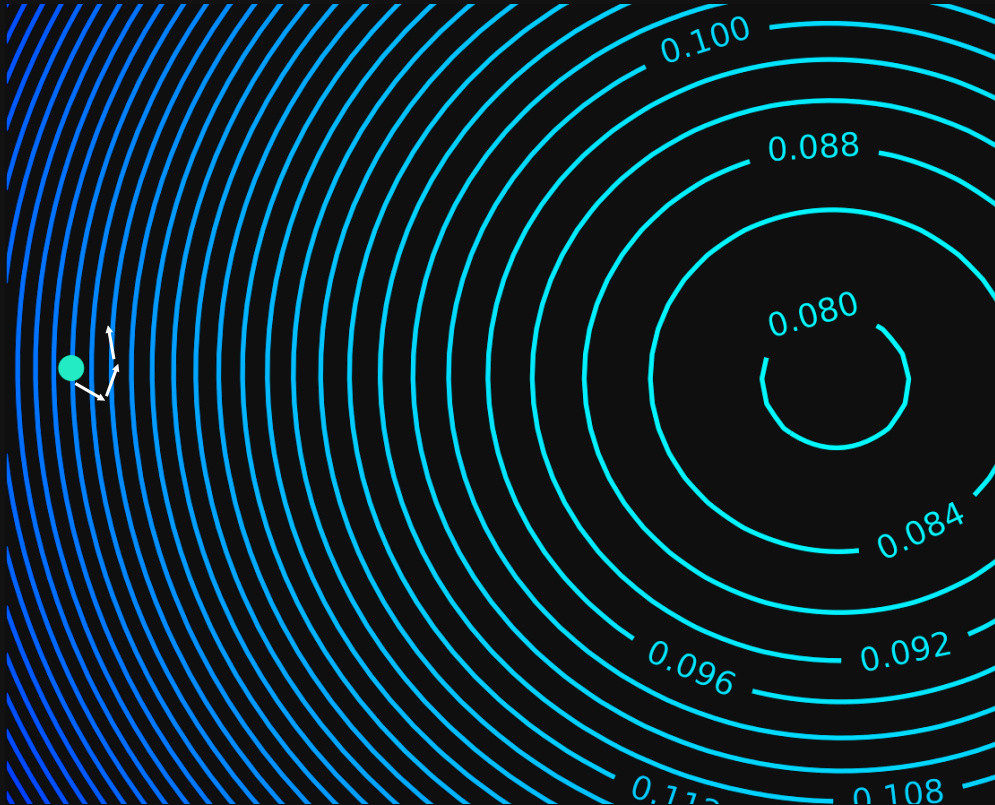
Stochastic Gradient Descent

- At l -th iteration of Gradient descent
 - choose a random sample $(\mathbf{y}_i, \mathbf{f}_i)$ from the training set $\mathcal{S}$
 - compute the gradient of the loss function with respect to the model parameters θ_l using only this sample

$$\theta_{l+1} = \theta_l - \eta_l \nabla_{\theta} J_i(\theta_l)$$

- Converges to a local minimum of the loss function

Stochastic Gradient Descent



Problems with stochastic gradient descent:

- similar to gradient descent
- fast per iteration, but requires many iterations to converge
- high variance (noisy updates)

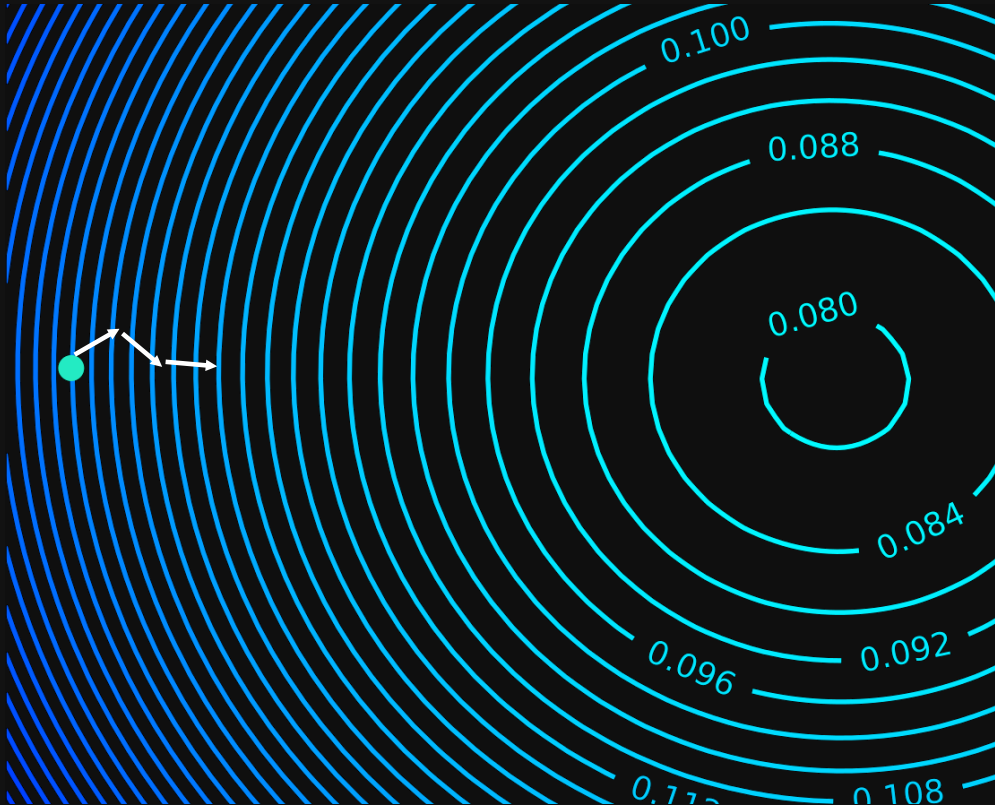
Mini-Batch Stochastic Gradient Descent

- Randomly shuffle training set $\mathcal{S}$ into mini-batches $\mathcal{S}_j$ of size n
- At l -th iteration of Stochastic Gradient descent
 - select a mini-batch $\mathcal{S}_l$
 - compute the gradient of the loss function with respect to the model parameters θ_l using only this batch

$$\theta_{l+1} = \theta_l - \eta_l \sum_{j \in \mathcal{S}_j} \nabla_{\theta} J_j(\theta_l)$$

- After $\frac{N}{n}$ iterations: 1 Epoch (all samples in the training set have been used once)
- Reshuffle after each epoch

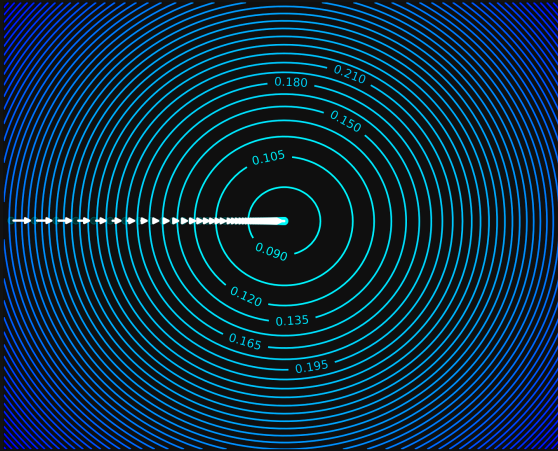
Mini-Batch Gradient Descent



Observations:

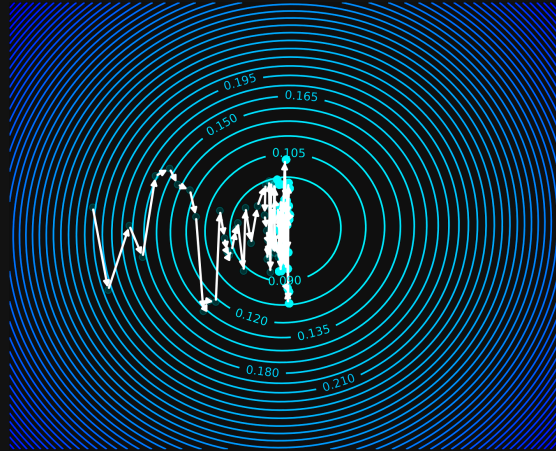
- similar to stochastic gradient descent
- faster convergence than stochastic gradient descent
- less noisy updates

(Stochastic) Gradient Descent



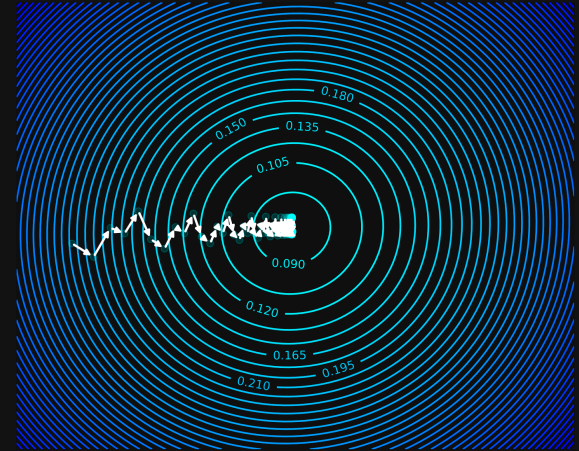
Gradient Descent

- `batch_size = N`
- 1 epoch = 1 iteration



Stochastic Gradient Descent

- `batch_size = 1`
- 1 epoch = N iterations



Mini-Batch Gradient Descent

- `batch_size = n`
- 1 epoch = N/n iterations

Adam Optimizer

- Adaptive Moment Estimation (Adam) is an extension of stochastic gradient descent that computes adaptive learning rates for each parameter
- Combines the advantages of two other extensions of SGD: AdaGrad and RMSProp
- Maintains two moving averages for each parameter:
 - **First moment** (mean): $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
 - **Second moment** (uncentered variance): $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} m_t$$

- typically, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$

Computation of Gradients: Backpropagation

- For all optimizers: gradients $\nabla_{\theta} J(\theta)$ need to be computed: **backpropagation algorithm**

- Recall the MLP architecture ($l = 1, \dots, L$)

$$z_i^l = \sum_{j=1}^{d_{l-1}} w_{ij}^l a_j^{l-1} + b_i^l$$

$$a_i^l = \sigma(z_i^l)$$

$$\hat{y}_i = \sigma^L(z_i^L) = f_{\theta}^*(\mathbf{y}_i)$$

- Chain rule (backward direction):

$$\begin{aligned} \frac{\partial J}{\partial z_i^l} &= \sum_{k=1}^{d_{l+1}} \frac{\partial J}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial z_i^l} = \sum_{k=1}^{d_{l+1}} \frac{\partial J}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial a_i^l} \frac{\partial a_i^l}{\partial z_i^l} \\ &= \sum_{k=1}^{d_{l+1}} \frac{\partial J}{\partial z_k^{l+1}} w_{ki}^{l+1} \sigma'(z_i^l) \end{aligned}$$

- Chain rule for derivatives:

$$\frac{\partial J}{\partial w_{ij}^l} = \frac{\partial J}{\partial z_i^l} \frac{\partial z_i^l}{\partial w_{ij}^l} = \frac{\partial J}{\partial z_i^l} a_j^{l-1}$$

$$\frac{\partial J}{\partial b_i^l} = \frac{\partial J}{\partial z_i^l} \frac{\partial z_i^l}{\partial b_i^l} = \frac{\partial J}{\partial z_i^l}$$

- Recursive relationship:

$$\frac{\partial J}{\partial z_i^l} \equiv \delta_i^l = \sigma'(z_i^l) \sum_{k=1}^{d_{l+1}} \delta_k^{l+1} w_{ki}^{l+1}$$

$$\delta_i^L = \frac{\partial J}{\partial \hat{y}_i} \sigma'^L(z_i^L)$$

Computation of Gradients: Matrix Multiplications and Autodiff

- Backpropagation can be efficiently implemented using matrix multiplications
- Forward pass:
 - For each layer l :
 - compute $\mathbf{z}^l = \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l$, where $\mathbf{W}^l \in \mathbb{R}^{d_l \times d_{l-1}}$ and $\mathbf{a}^{l-1} \in \mathbb{R}^{d_{l-1}}$ and $\mathbf{b}^l \in \mathbb{R}^{d_l}$
 - compute $\mathbf{a}^l = \sigma(\mathbf{z}^l)$
 - compute and cache $\sigma'(\mathbf{z}^l)$ using automatic differentiation
- Backward pass:
 - compute $\delta^L = \frac{\partial J}{\partial \mathbf{y}} \mathbb{I} \sigma'^L(\mathbf{z}^L)$
 - For each layer l :
 - compute $\delta^l = \sigma'(\mathbf{z}^l) \odot \left(\mathbf{W}^{l+1T} \delta^{l+1} \right)$, where $\odot$ is the element-wise product
 - compute $\frac{\partial J}{\partial \mathbf{W}^l} = \delta^l \mathbf{a}^{l-1T}$ and $\frac{\partial J}{\partial \mathbf{b}^l} = \delta^l$

Mini-Batch Version: More Matrices

- n^l : number of neurons in layer l
- m : number of samples in the mini-batch
- $\mathbf{X} \in \mathbb{R}^{n^0 \times m}$: input data matrix
- $\mathbf{W}^l \in \mathbb{R}^{n^l \times n^{l-1}}$: weight matrix for layer l
- $\mathbf{b}^l \in \mathbb{R}^{n^l}$: bias vector for layer l (broadcasted)
- $\mathbf{Z}^l \in \mathbb{R}^{n^l \times m}$: weighted input matrix for layer l
- $\mathbf{A}^l \in \mathbb{R}^{n^l \times m}$: activation matrix for layer l
- $\Delta^l \in \mathbb{R}^{n^l \times m}$: error matrix for layer l

Pass	Operation	Shape(s)
Forward	$\mathbf{Z}^l = \mathbf{W}^l \mathbf{A}^{l-1} + \mathbf{b}^l$	$[n^l \times m] = [n^l \times n^{l-1}] \cdot [n^{l-1} \times m]$
	$\mathbf{A}^l = \sigma(\mathbf{Z}^l)$	element-wise operation $[n^l \times m]$
Backward	$\Delta^l = \sigma'(\mathbf{Z}^l) \odot \mathbf{W}^{l+1T} \Delta^{l+1}$	$[n^l \times m] = [n^l \times m] \odot [n^l \times n^{l+1}] \times [n^{l+1} \times m]$
	$\frac{\partial J}{\partial \mathbf{W}^l} = \Delta^l \mathbf{A}^{l-1T}, \frac{\partial J}{\partial \mathbf{b}^l} = \Delta^l$	$[n^l \times n^{l-1}] = [n^l \times m] \cdot [m \times n^{l-1}]$

🤔 But Why Tensors, Then?

Even if MLPs only use matrices...

- **Frameworks generalize beyond MLPs:**
 - CNNs: 4D tensors → (batch, channels, height, width)
 - RNNs/Transformers: 3D+ tensors → (batch, sequence, embedding)
- **Auto-diff engines** work with arbitrary-rank tensors
- Tensors = uniform abstraction for:
 - Batched computation
 - Broadcasting
 - GPU acceleration

Deep Learning Frameworks

- Simplify the implementation of neural networks
- Provide high-level APIs for building, training, and evaluating models
- Support for automatic differentiation
- Optimized matrix operations and GPU acceleration



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich

PyTorch Overview

The Fundamental Building Block: `torch.Tensor`

```
1  import torch
2
3  # Create a 2D tensor
4  x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
5
6  # Random weight tensor on GPU
7  w = torch.randn(2, 2).to("cuda")
8
9  # Enable autograd tracking
10 x = torch.tensor([[1.0, 2.0]], requires_grad=True)
11
12 # Forward computation (tracked)
13 y = x @ w.T
14
15 # Stop tracking: detach from graph
16 y_detached = y.detach()
17
18 # You can check:
19 print(y.requires_grad)      # True
20 print(y_detached.requires_grad) # False
```

- Like NumPy arrays, but with GPU support and autograd.
- `.to("cuda")` Moves tensor to the GPU for faster computation (if available).
- `requires_grad=True` Enables gradient tracking for optimization. Needed for model parameters.
- `.detach()` Returns a new tensor that shares data but is not part of the computation graph. Use when you want to stop gradients (e.g. logging or freezing layers).

Define a Model: Subclass `nn.Module`

```
1 # Simpler: use nn.Sequential
2 import torch.nn as nn
3
4 model = nn.Sequential(
5     nn.Linear(4, 8),
6     nn.ReLU(),
7     nn.Linear(8, 1)
8 )
```

- `nn.Module` helps manage parameters and layers cleanly
- `forward()` defines the computation graph
- Use `nn.Sequential` for simple, linear stacks of layers
 - no need to define `forward()` explicitly

A Layer is a Module Too: `nn.Linear`

```
1 layer = nn.Linear(4, 8)
2
3 x = torch.randn(1, 4) # batch size = 1, features = 4
4 out = layer(x)
5
6 print(out.shape) # torch.Size([1, 8])
```

Automatically includes weights + biases, and supports gradients.

The Forward Pass + Autograd

```
1  model = SimpleMLP()
2  x = torch.randn(5, 4)      # mini-batch of 5 samples
3  out = model(x)             # forward pass
4  loss = out.sum()           # dummy loss
5
6  loss.backward()            # computes dLoss/dParam
```

Gradients are now stored in each parameter's `.grad` field.

Inspecting Model Parameters

```
1  # Get all parameters
2  for param in model.parameters():
3      print(param.shape)
4
5  # Named parameters
6  for name, param in model.named_parameters():
7      print(name, param.shape)
8
9  torch.save(model.state_dict(), "model.pt")
10 model.load_state_dict(torch.load("model.pt"))
```

Use `.state_dict()` for fine control and reproducibility.

torch.nn.functional vs torch.nn

torch.nn

Layer objects with state

```
1 self.relu = nn.ReLU()
2 self.linear = nn.Linear(10, 5)
3
4 x = self.linear(x)
5 x = self.relu(x)
```

- Registered as part of the model
- Has internal weights, tracked in `.parameters()`

torch.nn.functional

Stateless ops

```
1 import torch.nn.functional as F
2
3
4 x = F.linear(x, weight, bias) # pass weights manually
5 x = F.relu(x)
```

- Not registered in the model
- Good for custom layers or one-off ops

Best Practice

Use nn for model structure, F.* for flexibility and lower-level control.

Training Step: `optimizer.step()`

```
1  optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
2
3  # Dummy input/output
4  x = torch.randn(5, 4)
5  y = torch.randn(5, 1)
6
7  # Training step
8  out = model(x)
9  loss = nn.functional.mse_loss(out, y)
10 loss.backward()
11 optimizer.step()
12 optimizer.zero_grad()
```

`.step()` updates weights, `.zero_grad()` resets gradients.

Inference in PyTorch: Use `torch.no_grad()`

```
1 model.eval() # important: set to inference mode
2
3 with torch.no_grad():
4     x = torch.randn(1, 4)
5     out = model(x)
```

- Disables autograd inside the block
- No computation graph is created
- Reduces memory + improves speed
- `model.eval()` disables Dropout, BatchNorm updates

Exclude from Computational Graph: `.detach()`

```
1 output = model(x).detach()
```

- Removes tensor from the computation graph
- Still useful outside of `no_grad()`
- Great for logging, visualizations, etc.

Datasets: `torch.utils.data.Dataset`

A `Dataset` is a read-only wrapper around your data

- Must implement:
 - `__len__()` — number of samples
 - `__getitem__(index)` — returns a single data point

```
1 from torch.utils.data import Dataset
2
3 class MyDataset(Dataset):
4     def __init__(self, data):
5         self.data = data
6
7     def __getitem__(self, idx):
8         return self.data[idx]
9
10    def __len__(self):
11        return len(self.data)
```

This is the lowest-level API for loading datasets in PyTorch.

DataLoader : Batching, Shuffling, Parallel Loading

A `DataLoader` wraps a `Dataset` to enable:

- Mini-batches
- Shuffling
- Multi-process data loading
- Pinning memory (for fast GPU transfer)

```
1  from torch.utils.data import DataLoader
2
3  loader = DataLoader(
4      my_dataset,
5      batch_size=32,
6      shuffle=True,
7  )
8
9  for batch in loader:
10     process(batch)
```

Use `DataLoader` in every training loop — it handles batching and efficiency for you.

Recap

- `Tensor` → basic data container (with autograd!)
- `nn.Module` → base class for models
- `nn.Linear` → layers that manage weights
- `forward()` → defines computation
- `loss.backward()` + `optimizer.step()` → training
- `torch.no_grad()` → inference mode
- `Dataset` + `DataLoader` → data handling

```
1 x = torch.tensor([1.0], requires_grad=True)
2 y = x * 2
3 y.backward()
4 print(x.grad) # tensor([2.])
```

User code looks and feels like native Python, but is backed by a powerful engine underneath

Under the Hood: PyTorch Execution Model

Layer	Description
Python Frontend	You write code using <code>torch.*</code> APIs
Autograd Engine	Tracks ops & builds computation graph
ATen	Core C++ tensor library
C++/CUDA Kernels	Low-level execution (CPU, GPU)
XLA, Vulkan, etc	Alternate hardware targets

- Operations are dispatched via the **dispatcher** to the appropriate backend (e.g., CPU/GPU).
- PyTorch uses a dynamic computation graph (define-by-run) with native code under the hood.

`torch.compile()` — Ahead-of-Time (AOT) Mode

Introduced in **PyTorch 2.0+**

- Compiles model to a graph (speed)
- Wraps model in an optimizing runtime
- Uses TorchDynamo + backend compiler (e.g. TorchInductor)
 - TorchInductor uses Triton to generate fused GPU kernels
- Works with most models & training loops

```
1 compiled_model = torch.compile(model)
2 out = compiled_model(input)
```

Profiling in PyTorch

Use PyTorch's built-in profiler to analyze bottlenecks in training and inference.

Basic Usage

```
1  import torch.profiler
2
3  with torch.profiler.profile(
4      activities=[
5          torch.profiler.ProfilerActivity.CPU,
6          torch.profiler.ProfilerActivity.CUDA,
7      ],
8      record_shapes=True,
9      profile_memory=True
10 ) as prof:
11     model(input)
12
13 print(prof.key_averages().table(
14     sort_by="cuda_time_total"))
```

Output Options

- Terminal summary
- Chrome trace viewer
(`prof.export_chrome_trace("trace.json")`)
- TensorBoard plugin
(`torch.profiler.tensorboard_trace_handler`)

NVIDIA Profiling Tools

Tool	Purpose
Nsight Systems	Timeline of CPU/GPU interaction
Nsight Compute	Kernel-level GPU performance
nvprof	Lightweight CLI profiler (legacy)
ncu / nsys	Modern CLI tools

Run PyTorch with `nsys` (Nsight Systems)

```
1 nsys profile -o output_report python train.py
```

- Collects timeline, kernel launches, CPU–GPU syncs
- Outputs `.qdrep` or `.nsys-rep` for use in GUI

Optional flags:

```
1 --capture-range=cudaProfilerApi \  
2 --trace=cuda,osrt,nvtx
```

Tip

Recommended: add NVTX annotations for custom markers

Add NVTX Markers in Your Code

PyTorch supports NVTX via `torch.cuda.nvtx`

```
1  import torch.cuda.nvtx as nvtx
2
3  nvtx.range_push("forward pass")
4  out = model(x)
5  nvtx.range_pop()
6
7  nvtx.range_push("loss + backward")
8  loss = criterion(out, y)
9  loss.backward()
10 nvtx.range_pop()
```

- Tags your code with named regions
- Visible in Nsight Systems timeline
- Helps isolate bottlenecks: forward, backward, data loading, etc.

torchvision: Computer Vision

What it provides:

- Pretrained models (ResNet, ViT, Faster R-CNN, etc.)
- Common datasets (CIFAR, ImageNet, COCO...)
- Data transforms and augmentations
- Image utilities (`utils.draw_bounding_boxes` etc.)

```
1  from torchvision import models, transforms
2
3  model = models.resnet18(pretrained=True)
4  transform = transforms.Compose([
5      transforms.Resize(256),
6      transforms.CenterCrop(224),
7      transforms.ToTensor(),
8  ])
```

torchaudio: Audio & Speech

Tools for:

- Loading/writing audio (`torchaudio.load`)
- Preprocessing (spectrograms, MFCCs)
- Pretrained models (Wav2Vec2, HuBERT, etc.)
- ASR, speech commands, audio classification

```
1  import torchaudio
2
3  waveform, sr = torchaudio.load("clip.wav")
4  spec = torchaudio.transforms.MelSpectrogram()(waveform)
```

torchtext: NLP Datasets & Preprocessing



Includes:

- Text tokenizers and pipelines
- Ready-to-use NLP datasets (AGNews, IMDB, etc.)
- Pretrained embeddings (GloVe, fastText)

```
1  from torchtext.datasets import AG_NEWS
2
3  train_iter = AG_NEWS(split='train')
4  for label, text in train_iter:
5      print(label, text)
6      break
```

Best for traditional NLP workflows (before Transformers).

Hugging Face Transformers + PyTorch

Go-to library for **NLP**, **LLMs**, and increasingly **multimodal** deep learning.

What it provides:

- Pretrained models: BERT, GPT-2/3/4, LLaMA, Mistral, T5, etc.
- Tokenizers and model loading
- Pipelines for easy use
- Tools for fine-tuning and training



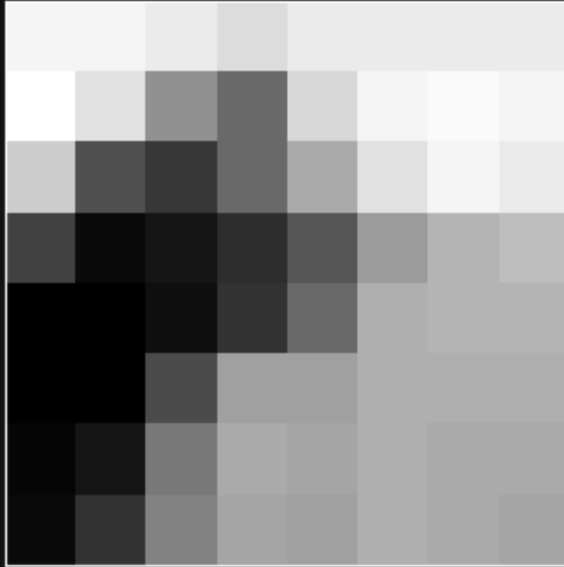
CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich

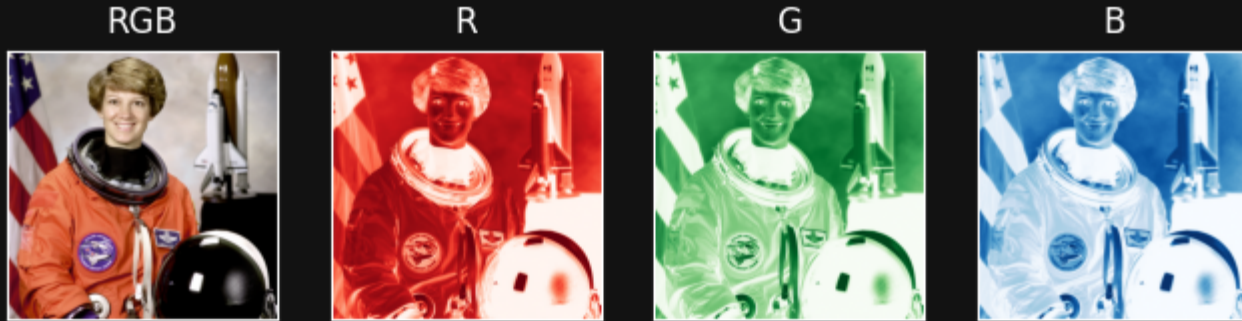
Working with Images

(Grayscale) Images are Matrices



203	202	195	185	196	197	196	195
209	187	128	101	179	204	205	203
173	81	60	98	146	187	201	197
71	28	37	55	84	137	155	163
20	19	32	58	100	150	156	156
20	19	76	139	138	152	152	151
26	35	110	149	144	151	147	145
27	60	118	142	139	149	146	144

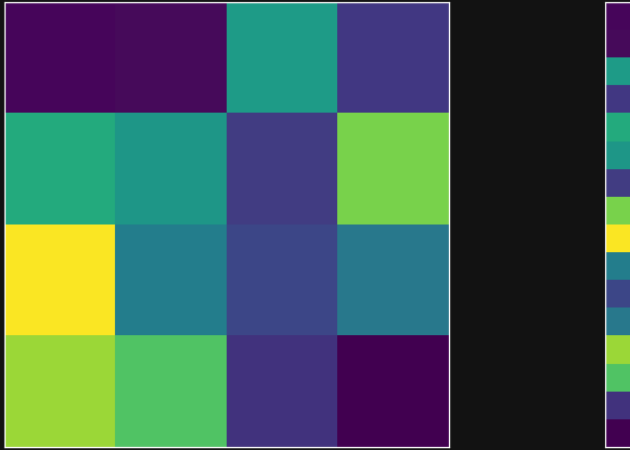
(Color) Images are Tensors



PyTorch: torchvision.transforms.v2s (V2)

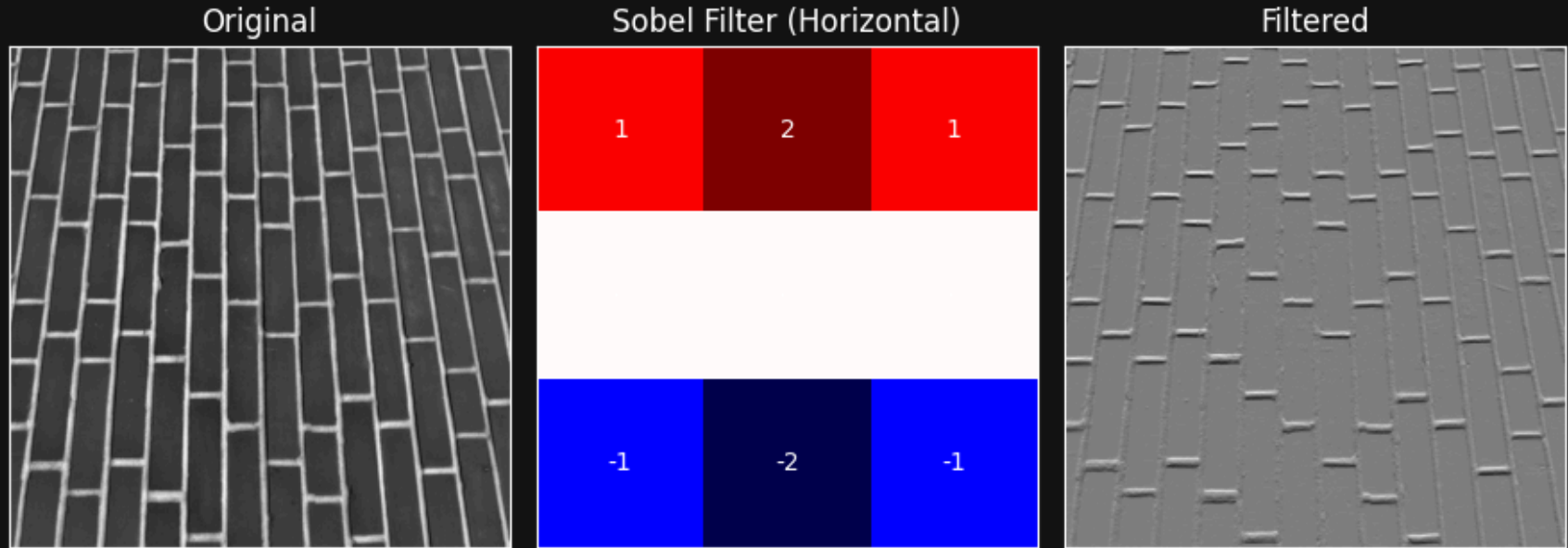
```
1 import torchvision.transforms.v2 as transforms
2
3 transform = transforms.Compose([
4     # Convert PIL image to tensor
5     transforms.ToImage(),
6     # Apply other transforms
7     # Convert to float32 tensor (scale to range [0,1])
8     transforms.ToDtype(torch.float32, scale=True),
9     # Apply normalization
10    transforms.Normalize(
11        mean=[0.485, 0.456, 0.406],
12        std=[0.229, 0.224, 0.225]
13    )
14 ])
```

Images: 1D vs 2D Representation

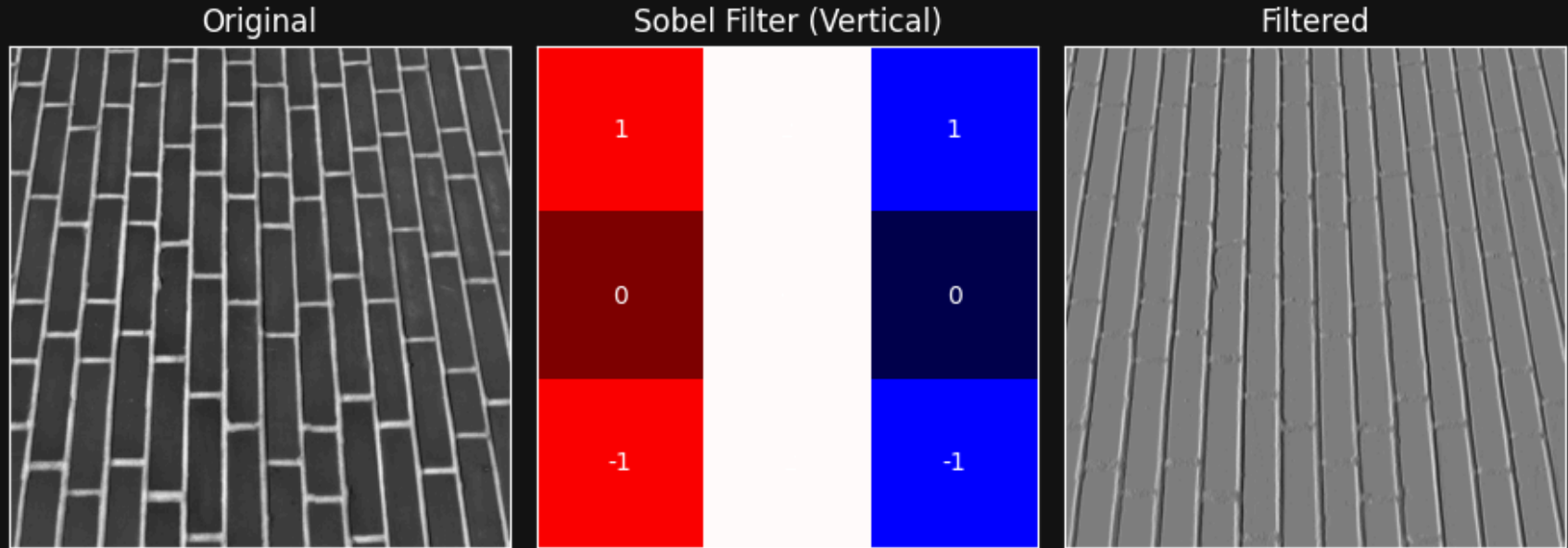


How can *spatial information* be *preserved* and *exploited*?

Convolutional Filters: Edge Detection



Convolutional Filters: Edge Detection



Convolution Operation

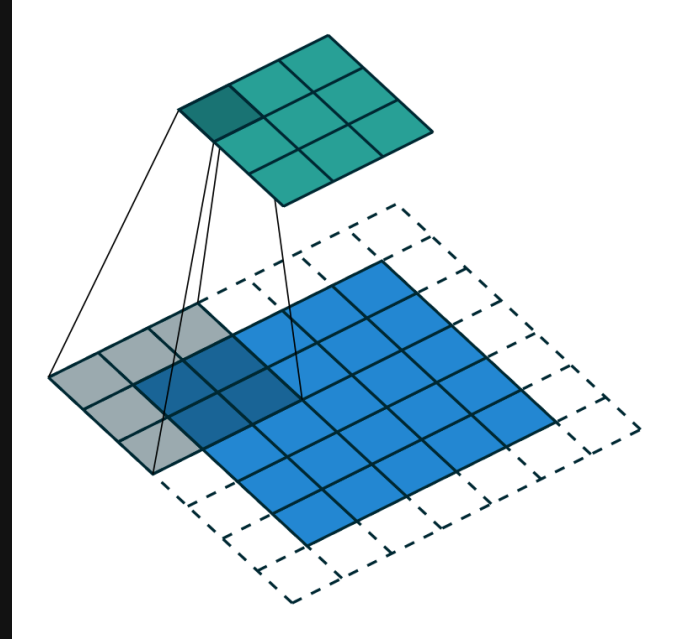
$$\begin{pmatrix} i_{11} & i_{12} \\ i_{21} & i_{22} \end{pmatrix} \star \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} = i_{11}w_{11} + i_{12}w_{12} + i_{21}w_{21} + i_{22}w_{22}$$

- Element-wise multiplication
- Summation

Convolution Operation: Visual Representation

0 ₀	0 ₁	0 ₂	0	0	0	0
0 ₂	3 ₂	3 ₀	2	1	0	0
0 ₀	0 ₁	0 ₂	1	3	1	0
0	3	1	2	2	3	0
0	2	0	0	2	2	0
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0

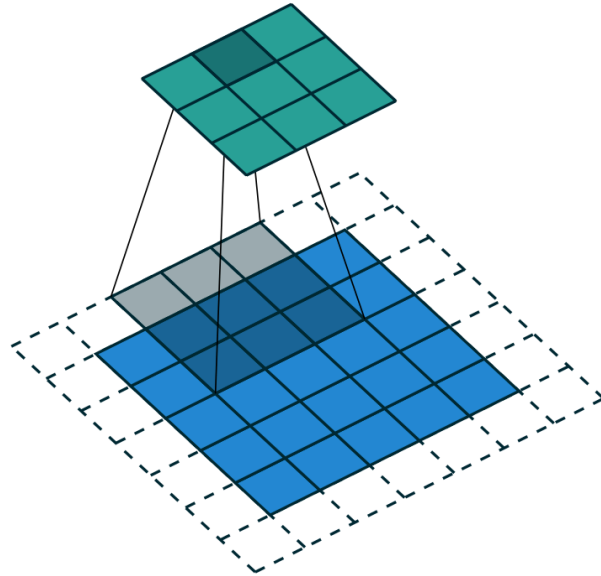


Dumoulin, Vincent, and Francesco Visin. "A guide to convolution arithmetic for deep learning." arXiv preprint arXiv:1603.07285 (2016).

Convolution Operation: Visual Representation

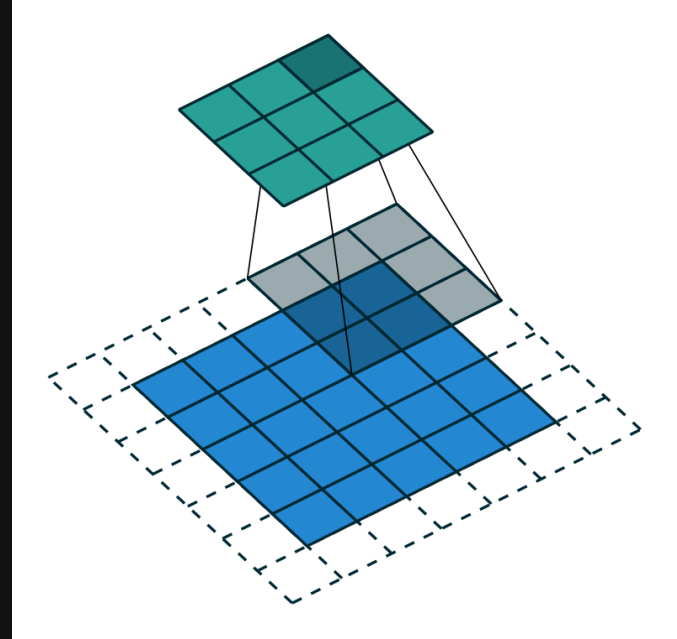
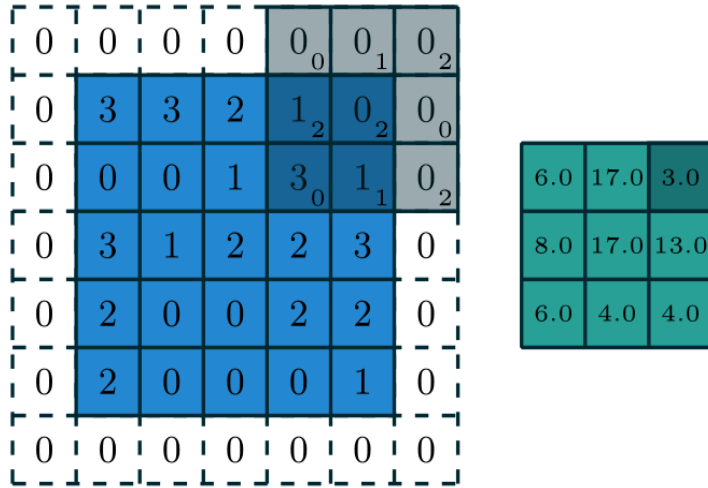
0	0	0 ₀	0 ₁	0 ₂	0	0
0	3	3 ₂	2 ₂	1 ₀	0	0
0	0	0 ₀	1 ₁	3 ₂	1	0
0	3	1	2	2	3	0
0	2	0	0	2	2	0
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0



Dumoulin, Vincent, and Francesco Visin. "A guide to convolution arithmetic for deep learning." arXiv preprint arXiv:1603.07285 (2016).

Convolution Operation: Visual Representation

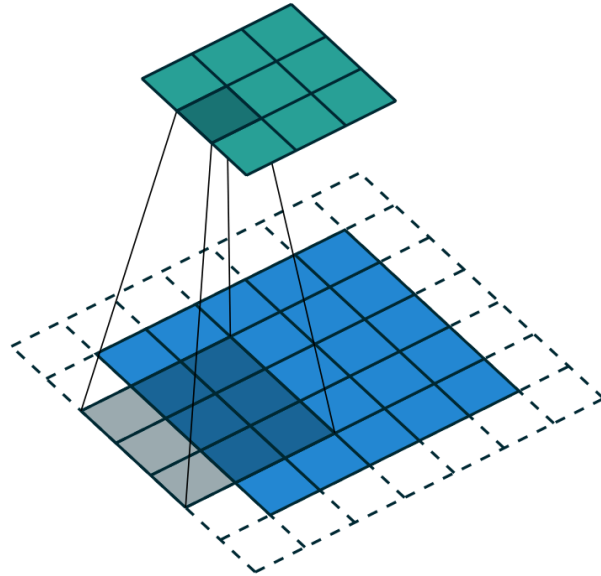


Dumoulin, Vincent, and Francesco Visin. "A guide to convolution arithmetic for deep learning." arXiv preprint arXiv:1603.07285 (2016).

Convolution Operation: Visual Representation

0	0	0	0	0	0	0
0	3	3	2	1	0	0
0 ₀	0 ₁	0 ₂	1	3	1	0
0 ₂	3 ₂	1 ₀	2	2	3	0
0 ₀	2 ₁	0 ₂	0	2	2	0
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	17.0	3.0
8.0	17.0	13.0
6.0	4.0	4.0



Dumoulin, Vincent, and Francesco Visin. "A guide to convolution arithmetic for deep learning." arXiv preprint arXiv:1603.07285 (2016).

PyTorch: 2D Convolution

$$O(N_i, C_{O_j}) = b(C_{O_j}) + \sum_{k=0}^{C_I-1} W(C_{O_j}, k) \star I(N_i, k)$$

```
1 torch.nn.Conv2d(  
2     in_channels,  
3     out_channels,  
4     kernel_size,  
5     stride=1, padding=0, dilation=1,  
6     bias=True,  
7     padding_mode='zeros'  
8 )
```

`padding="same"` automatically adds padding to keep the output size the same as the input size.



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich

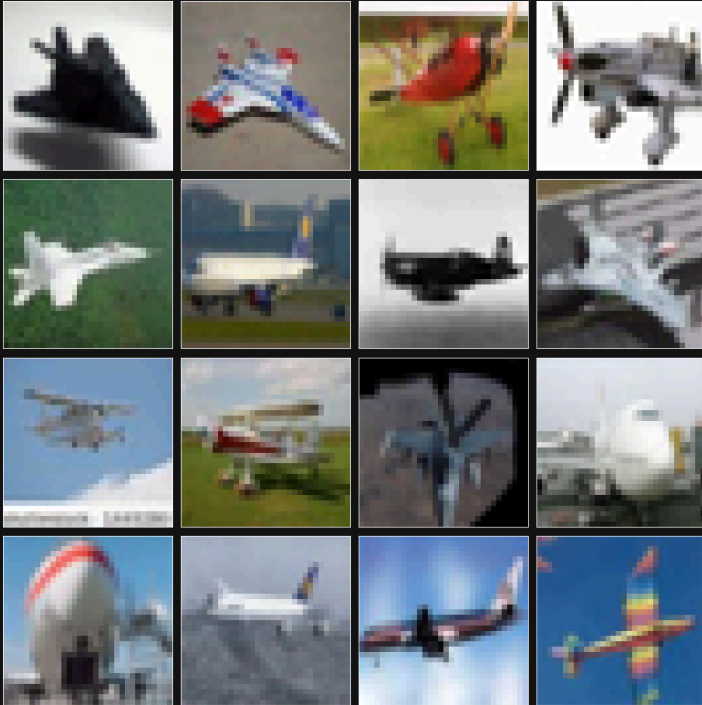
Convolutional Neural Networks (CNNs)

Computer Vision

Computer vision is an interdisciplinary field that deals with how computers can be made to gain high-level understanding from digital images or videos. From the perspective of engineering, it seeks to automate tasks that the human visual system can do.

- Image classification
- Object detection
- Self-driving cars/autonomous vehicles
- Medicine and healthcare
- Generative art
- ...

Challenges in Computer Vision



- Variability in appearance
 - Lighting
 - Angle
 - Occlusion
 - ...
- Different object scales and orientations
- Background clutter
- Limited labeled data
- ...

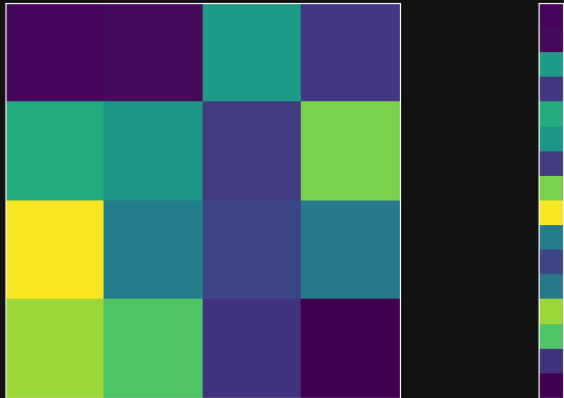
Fully Connected Neural Networks: How Many Parameters?

- Input: 100×100 pixel (grayscale image)
- First hidden layer: 1000 neurons

How many parameters (connections) are there between the input and first hidden layer?

$100 \times 100 \times 1000 = 10,000,000$ parameters!

Fully Connected Neural Networks: How Many Parameters?



- Too many parameters
- Loss of spatial information

How can *spatial information* be *preserved* and *exploited*?

We need Convolutional Neural Networks (CNNs)!

2D Convolutions

- Linear operation
- Weights shared across pixels
 - Reduces number of parameters
- An output pixel depends on neighboring input pixels
 - Retains spatial information

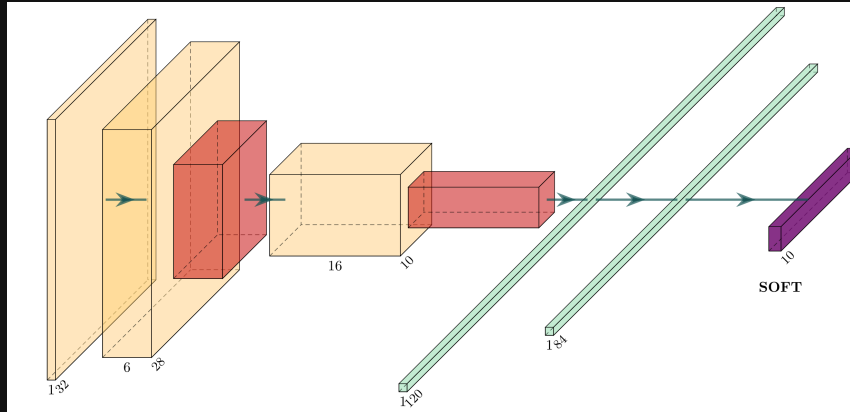
Convolutional Neural Networks: How Many Parameters?

- Input: 100×100 pixel, 3 color channels
- Convolutional kernel: 5×5
- Bias: true
- Output: 200 feature maps (100×100 pixels each)

How many parameters (connections) are there for the first layer?

$$(5 \times 5 \times 3 + 1) \times 200 = 15,200 \text{ parameters!}$$

CNN Ingredients



- Convolutional layers
- Pooling layers
- Activation functions
- Normalization layers
- Fully connected layers

CNNs for classification have two conceptual building blocks:

- Feature extractor (convolution layers)
- Classifier (linear layers)

<https://github.com/HarisIqbal88/PlotNeuralNet>

PyTorch: 2D Convolution

$$O(N_i, C_{O_j}) = b(C_{O_j}) + \sum_{k=0}^{C_I-1} W(C_{O_j}, k) \star I(N_i, k)$$

```
1 torch.nn.Conv2d(  
2     in_channels,  
3     out_channels,  
4     kernel_size,  
5     stride=1, padding=0, dilation=1,  
6     bias=True,  
7     padding_mode='zeros'  
8 )
```

`padding="same"` automatically adds padding to keep the output size the same as the input size.

PyTorch: Pooling

Max Pooling

```
1 torch.nn.MaxPool2d(  
2     kernel_size, stride=None,  
3     padding=0, dilation=1  
4 )
```

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

Average Pooling

```
1 torch.nn.AvgPool2d(  
2     kernel_size, stride=None,  
3     padding=0  
4 )
```

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

PyTorch: Pooling

Max Pooling

```
1 torch.nn.MaxPool2d(  
2     kernel_size, stride=None,  
3     padding=0, dilation=1  
4 )
```

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

Average Pooling

```
1 torch.nn.AvgPool2d(  
2     kernel_size, stride=None,  
3     padding=0  
4 )
```

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

PyTorch: Pooling

Max Pooling

```
1 torch.nn.MaxPool2d(  
2     kernel_size, stride=None,  
3     padding=0, dilation=1  
4 )
```

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

Average Pooling

```
1 torch.nn.AvgPool2d(  
2     kernel_size, stride=None,  
3     padding=0  
4 )
```

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

PyTorch: Pooling

Max Pooling

```
1 torch.nn.MaxPool2d(  
2     kernel_size, stride=None,  
3     padding=0, dilation=1  
4 )
```

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

Average Pooling

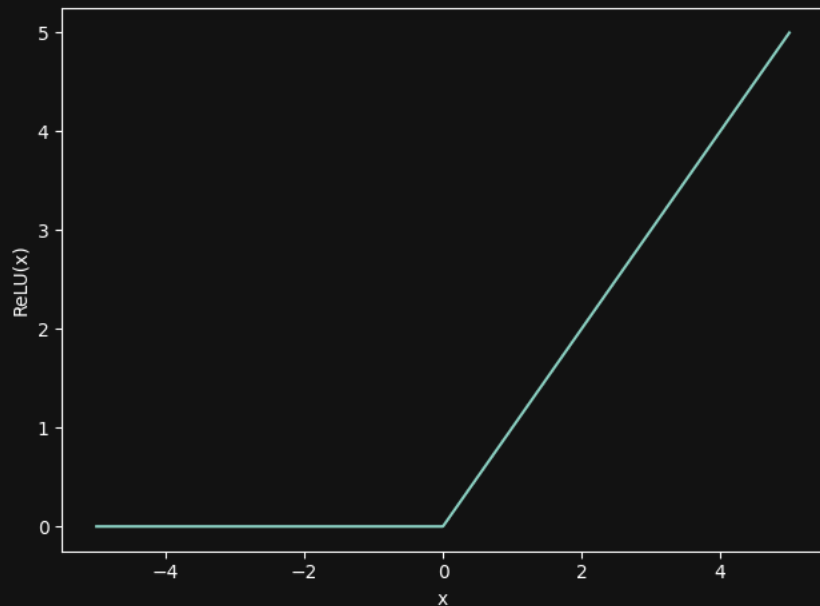
```
1 torch.nn.AvgPool2d(  
2     kernel_size, stride=None,  
3     padding=0  
4 )
```

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3

PyTorch: ReLU activation function

ReLU (Rectified Linear Unit) is a popular activation function in CNNs.



```
1 torch.nn.ReLU(inplace=False)
```

Functional API:

```
1 torch.nn.functional.relu(  
2     input,  
3     inplace=False  
4 )
```


Simple CNN in PyTorch

```
1  nn.Sequential([
2      nn.Conv2D(3, 16, 3),
3      nn.MaxPool2d(2),
4      nn.ReLU(),
5      nn.Conv2D(16, 32, 3),
6      nn.MaxPool2d(2),
7      nn.ReLU(),
8      nn.Flatten(),
9      nn.Linear(256, 10)
10 ])
```

PyTorch: `nn.Flatten` and `view`

`nn.Flatten()` is useful in conjunction with `nn.Sequential`. `torch.Tensor.view()` is more general (and used directly in `forward()`).

`nn.Flatten`

```
1 input = torch.randn(32, 1, 5, 5)
2 m = nn.Flatten(
3     start_dim=1,
4     end_dim=-1
5 )
6 output = m(input)
7 output.size()
8 # torch.Size([32, 25])
```

`torch.Tensor.view`

```
1 input = torch.randn(32, 1, 5, 5)
2 output = input.view(-1, 25)
3 output.size()
4 # torch.Size([32, 25])
```

CNN Zoo

Many CNN architectures:

- LeNet 5
- AlexNet (winner of ImageNet 2012)
 - Local response normalization
- VGG16/VGG19 (runner-up of ImageNet 2014)
- ResNet (winner of ImageNet 2015)
 - Batch normalization
 - Residual connections

Binary Classification

Classify images into two classes. The model predict the probability p_i of one class.

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N (y_i \log(p_i) + (1 - y_i) \log(1 - p_i))$$

```
1 import torch.nn.functional as F
2
3 def forward(self, x):
4     # ...
5     return F.sigmoid(x)
6
7 loss = nn.BCELoss()
```

```
1 import torch.nn.functional as F
2
3 def forward(self, x):
4     # ...
5     return self.linear_layer(x)
6
7 loss = nn.BCEWithLogitsLoss()
```

Multi-Class Classification: Probabilities

Classify images into multiple classes. The output of the model, a vector of N classes, can be converted to a probability distribution.

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^C e^{x_j}}$$

The model predict the probability p_{ij} for each class j of sample i .

Multi-Class Classification: Loss Function

Generalization of the binary cross-entropy loss function to multiple classes:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log(p_{ij})$$

```
1 import torch.nn.functional as F
2
3 def forward(self, x):
4     # ...
5     return self.linear_layer(x)
6
7 loss = nn.CrossEntropyLoss()
```

```
1 import torch.nn.functional as F
2
3 def forward(self, x):
4     # ...
5     return F.log_softmax(x, dim=1)
6
7 loss = nn.NLLLoss()
```

What have we missed?

- Classification and localization
 - Regression problem for bounding boxes
- Object detection
 - Classification and localization for multiple objects
- Semantic segmentation
 - Classification for each pixel (car, road, sky, etc.)
- Vision Transformers (ViT)
- ...

[LAB] CNN for Image Classification of CIFAR-10

Implement and train a CNN from scratch for image classification:

- Load and inspect the dataset
- Implement a simple CNN architecture
- Implement the training loop
- Train the CNN
- Evaluate the CNN performance



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

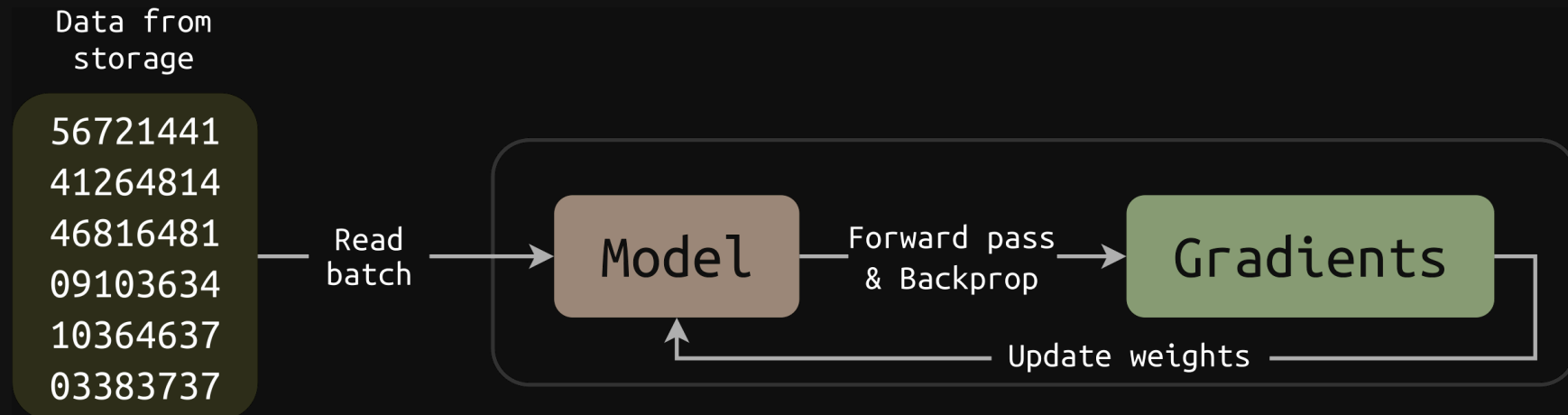
ETH zürich

Distributed Training

Why distributed training?

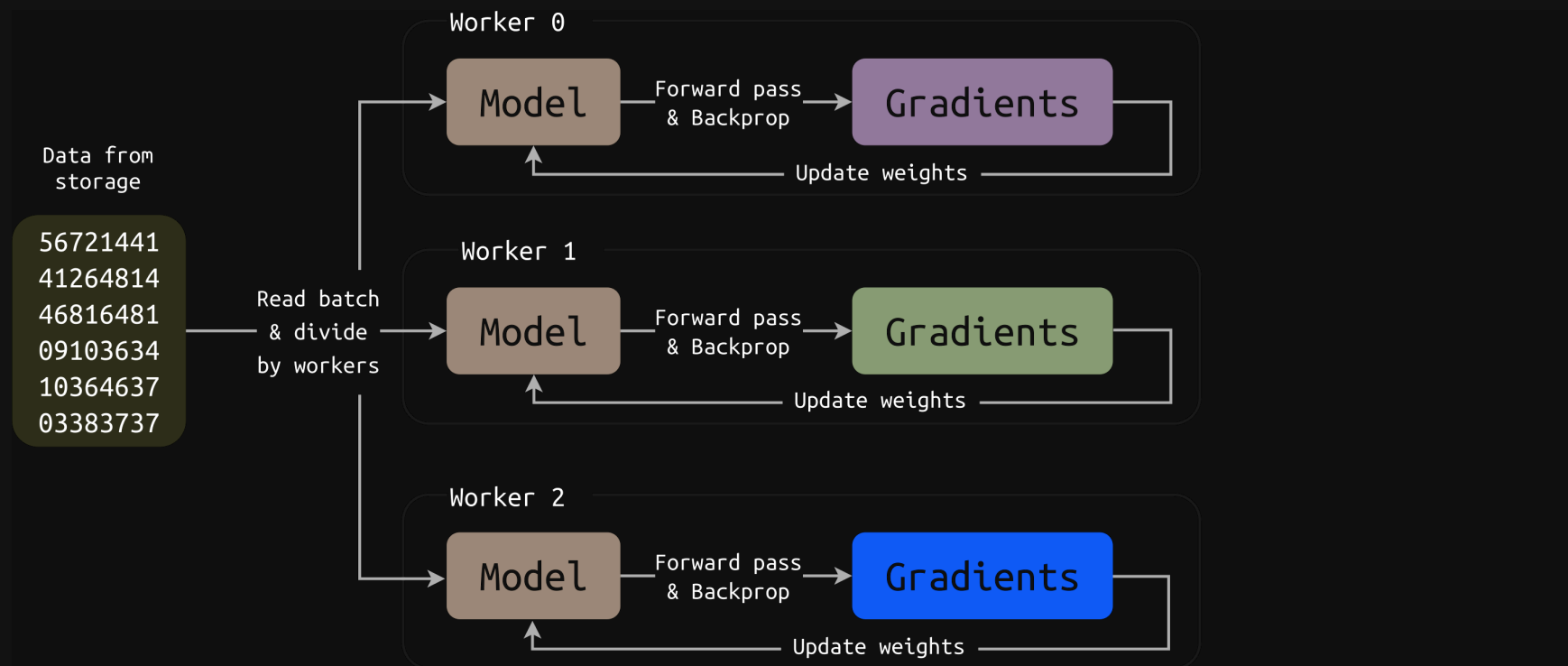
- Training deep neural networks is computationally expensive
- Models can be very large
 - e.g. Deepseek R1 has ~ 670 billion parameters
 - We need to store the model parameters in memory
 - Will not fit in a single GPU memory

Data Parallelism

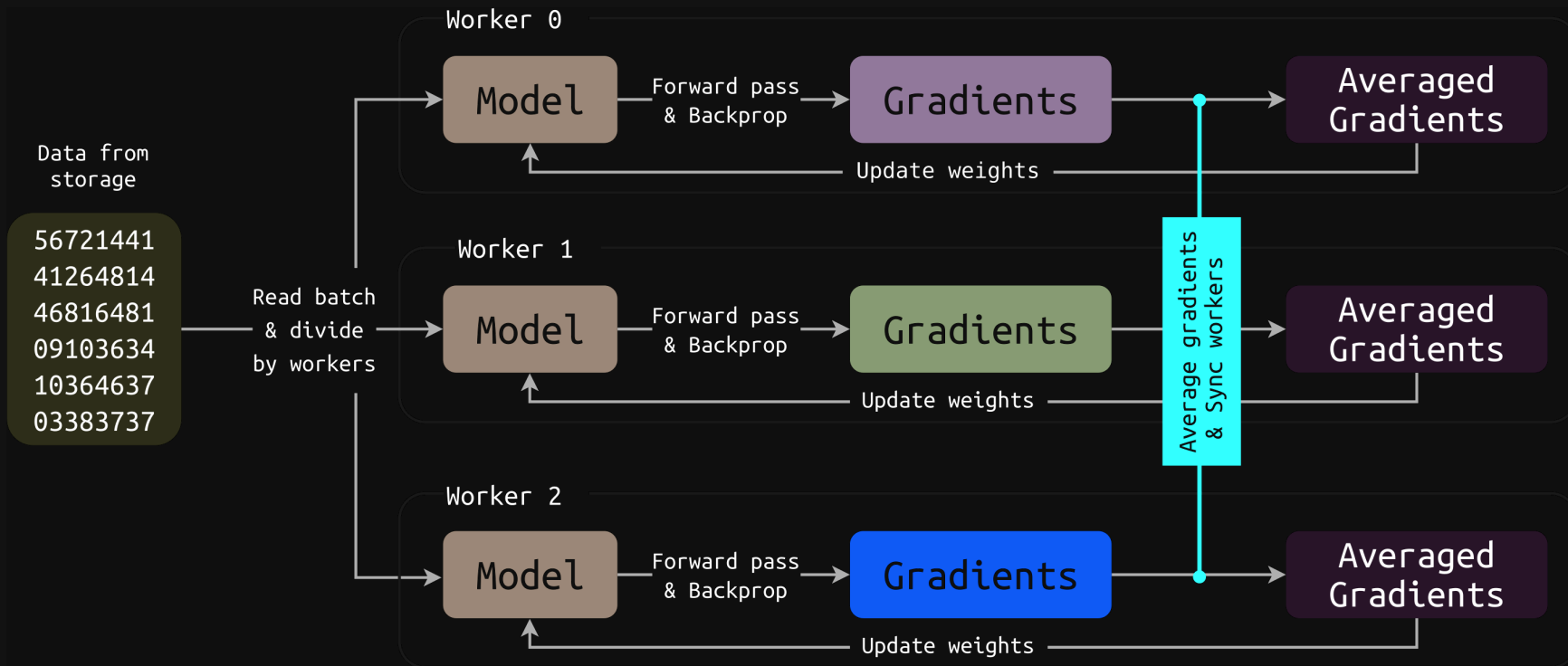


- Split the data across multiple GPUs
- Each GPU has a copy of the model
- Simplest way to distribute training: Distributed Data Parallel (DDP)

Data Parallelism: DDP



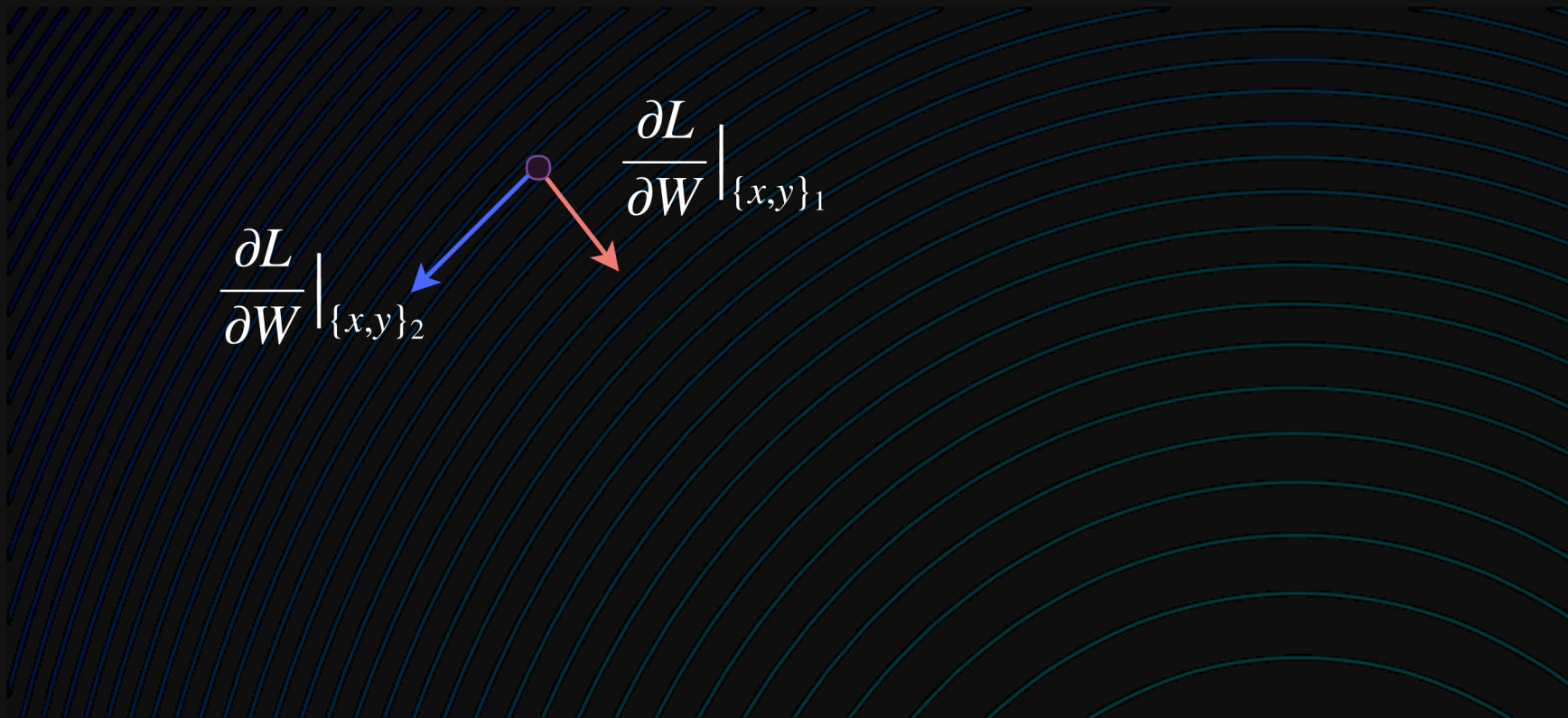
Data Parallelism: DDP

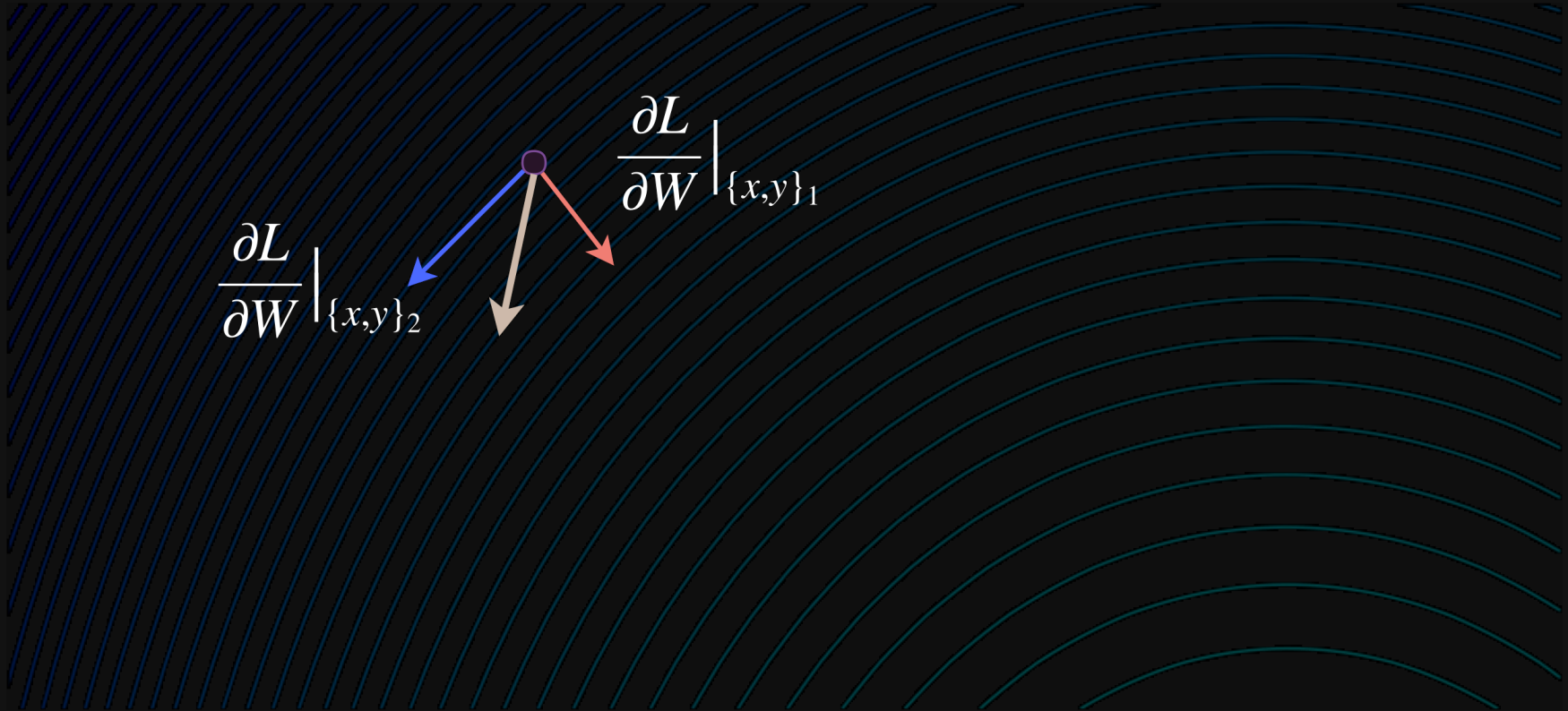


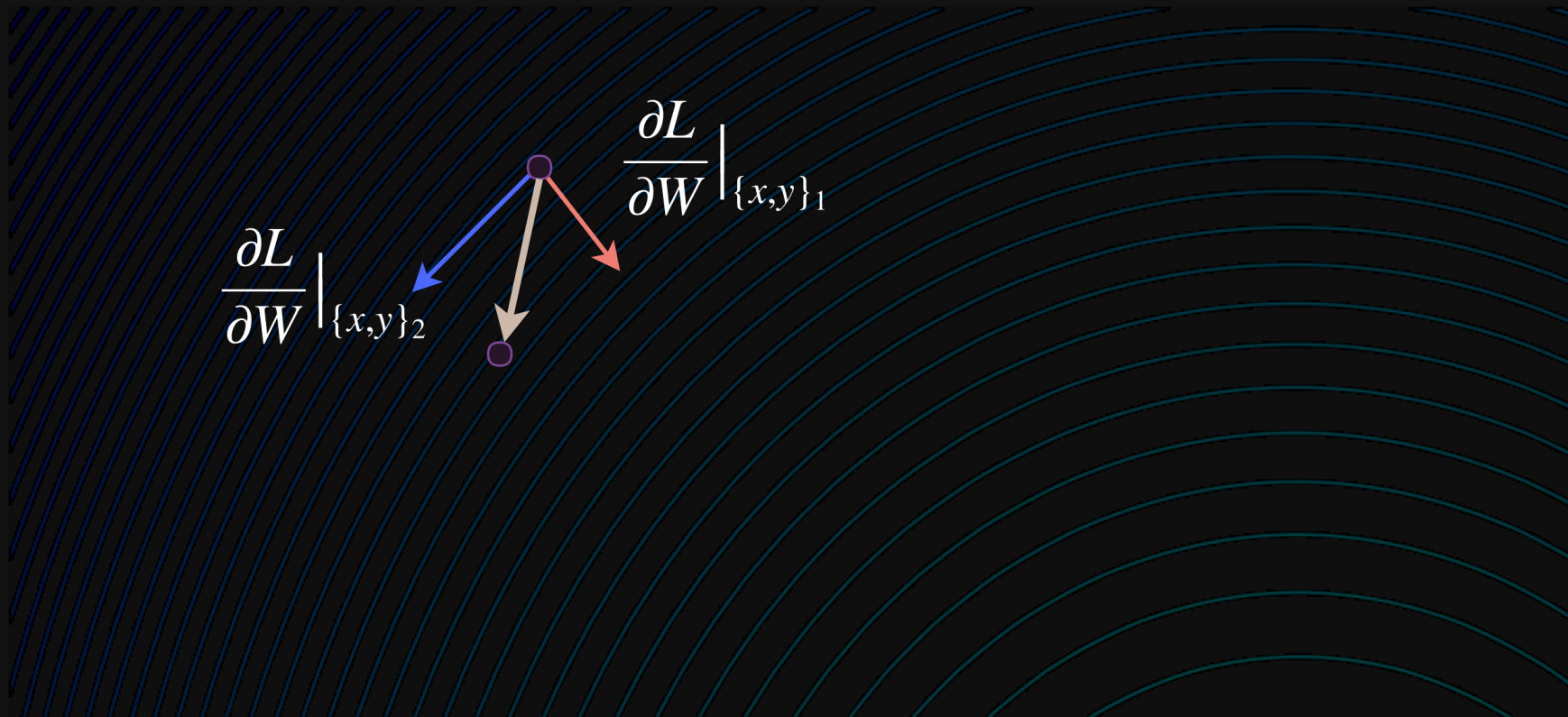


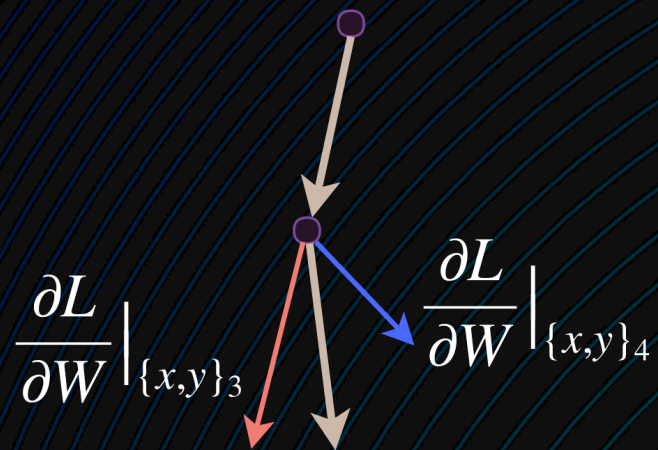


Worker 0
Worker 1









Data Parallelism: DDP

Pros:

- Easy to implement
- Scales well with the number of GPUs
- Can be used with any model
- Good computation/communication ratio
- Speedup is close to linear with the number of GPUs

Cons:

- Large models do not fit in a single GPU memory

Can we compress the model?

- Observations:
 1. Model states (in particular optimizer states) use the most memory
 2. Some states can be stored in lower precision (e.g. 16-bit floats)

Optimizer State: Adam

- Maintains two moving averages:

- **First moment** (mean) estimate

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\partial J}{\partial w_t}$$

- **Second moment** (uncentered variance) estimate

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\partial J}{\partial w_t} \right)^2$$

- Bias correction for both estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2}$$

- Update rule for weights:

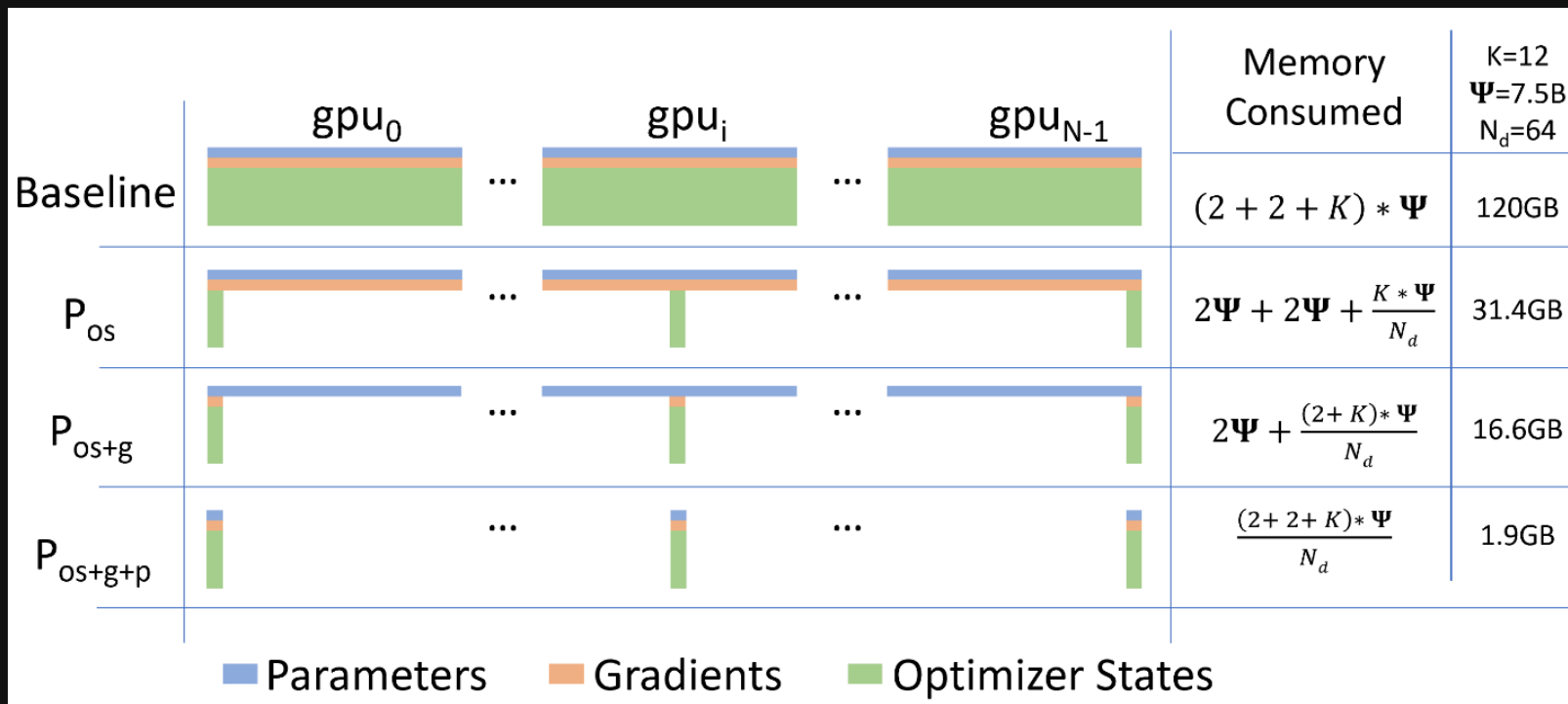
$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- Mixed precision training:

- Store gradients and weights in 16-bit floats
 - Use 32-bit floats for bias-corrected estimates during computation

Data Parallelism: Zero Redundancy Optimizer (ZeRO)

- Three stages of ZeRO:
 1. Optimizer State Partitioning (Pos)
 - 4x memory reduction
 - same communication volume
 2. Gradient Partitioning (Pos+g)
 - 8x memory reduction
 - same communication volume
 3. Parameter Partitioning (Pos+g+p)
 - Memory reduction is linear with data parallelism degree (N_d)
 - 1.5x increase in communication volume
- Developed by Microsoft Research (DeepSpeed)
- Additional features:
 - CPU offloading
 - Mitigates memory fragmentation



Rajbhandari, Samyam, et al. "Zero: Memory optimizations toward training trillion parameter models." SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2020.

- Ψ : Number of parameters
- K : Optimizer state memory overhead factor ($K = 12$ for Adam)

Fully Sharded Data Parallel (FSDP)

- PyTorch native implementation of ZeRO-3

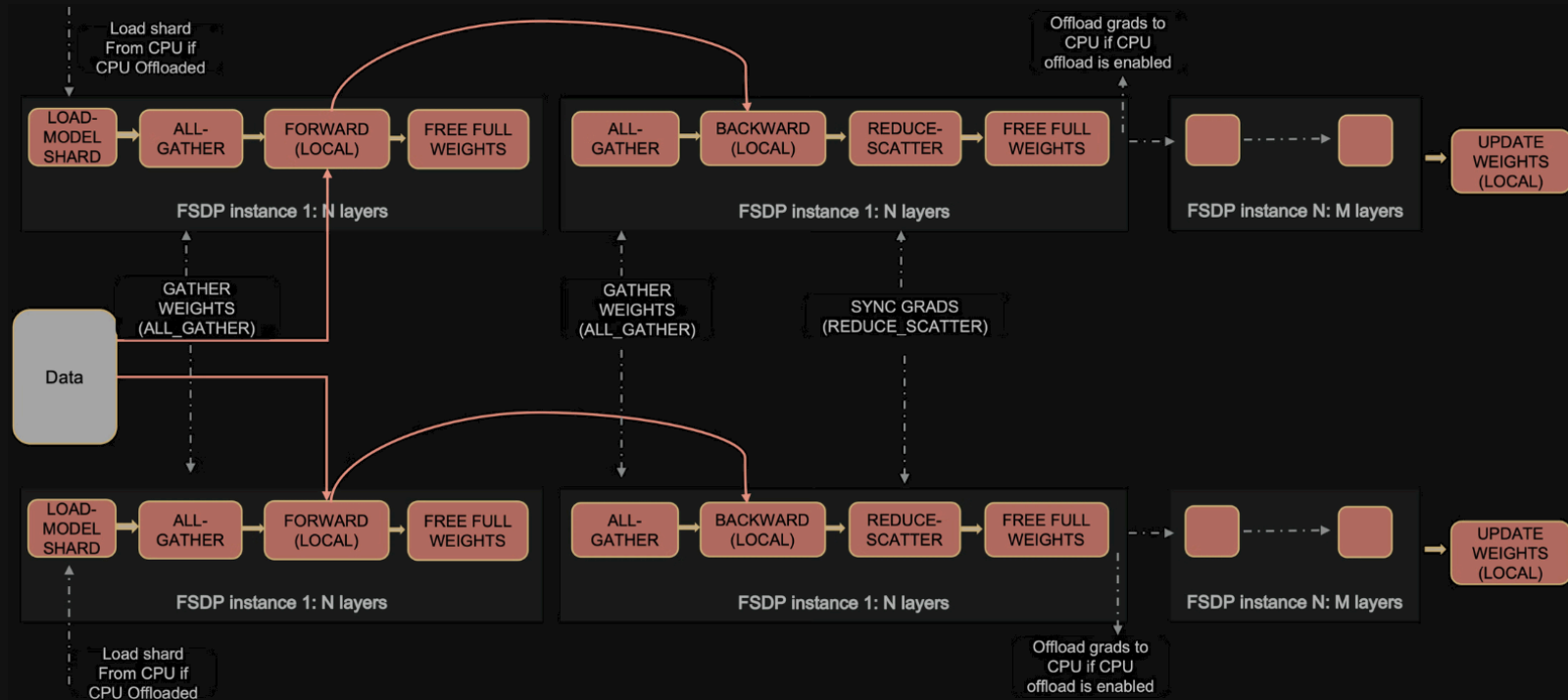
Forward Pass

- Each GPU holds **only its shard** of model weights
- Before computation, each layer's full weights are **gathered ("all-gather")** from shards
- Forward pass runs as usual
- Afterward, weights are optionally **discarded** to save memory

Backward Pass

- Gradients are computed **locally**
- Gradients are **reduced-scattered** across devices (each device keeps only the gradient shard it needs)
- Optimizer updates are done **shard-wise** (no full model needed)

Fully Sharded Data Parallel (FSDP)



<https://pytorch.org/blog/introducing-pytorch-fully-sharded-data-parallel-api/>

Data Parallelism: ZeRO/FSDP

Pros:

- Scales to large models
- Memory savings
- Supports mixed precision training
- Can be used with any model
- Good compute/communication ratio (1,5x DDP communication volume)

Cons:

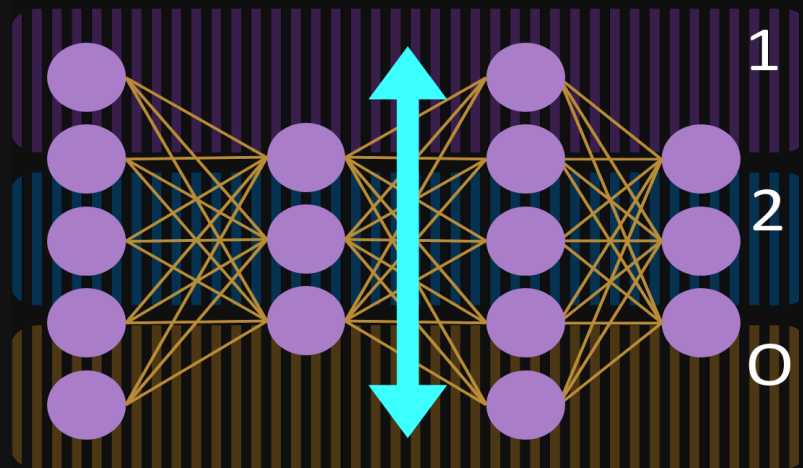
- More complex to implement & debug
- Overhead from frequent all-gather/reduce-scatter
- Very large layers:
 - may not fit in memory even with sharding
 - computation is not parallelized across GPUs

Model Parallelism: Tensor/Operator Parallelism

- Each GPU holds a part of the model
- Layers of the model are split across multiple GPUs
 - Not only data but also computation is split
 - Parallel (dense) linear algebra operations
- Requires more communication between GPUs
- Example: Megatron-LM, DeepSpeed, Axonn

Operator Parallelism

allreduce ring or neighbor communication



By Hoefler, Torsten, et al. "HammingMesh: A Network Topology for Large-Scale Deep Learning." *Communications of the ACM* 67.12 (2024): 97-105.

Model Parallelism: Tensor/Operator Parallelism

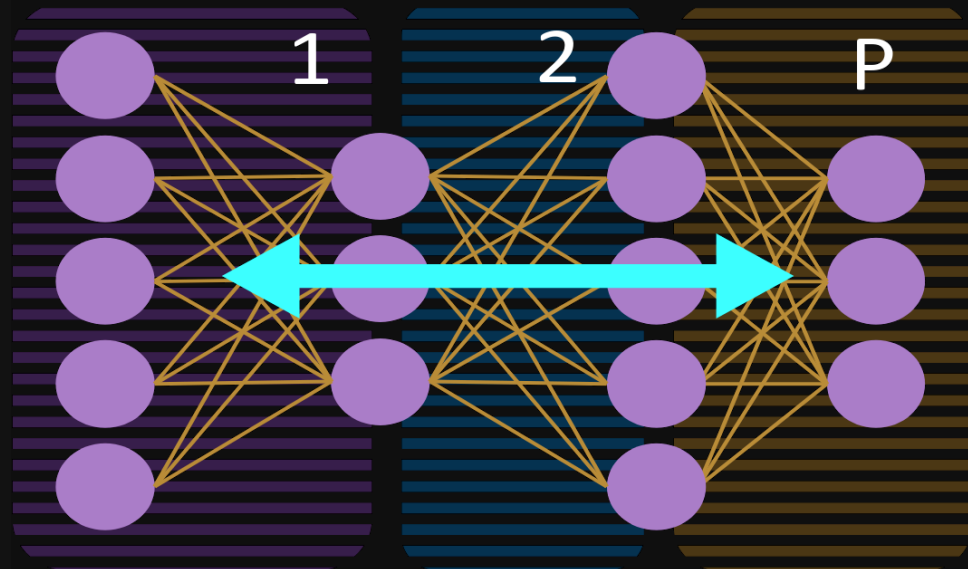
- Pros:
 - Can handle very large models that do not fit in a single GPU memory
 - Good for models with large layers (e.g. transformers)
- Cons:
 - More complex to implement: specialized frameworks needed
 - Only support a limited of distributed operators
 - often requires adapting the model
 - Requires more communication between GPUs
 - Requires low-latency fabric: Efficiency degrades quickly beyond a single node

Model Parallelism: Pipeline Parallelism

- Split the model into stages (groups of layers)
- Each stage is executed on a different GPU
- Use for very deep models (e.g. transformers)
- Communication:
 - Forward pass: send activations from one stage to the next
 - Backward pass: send gradients back through the stages
- Works with any model

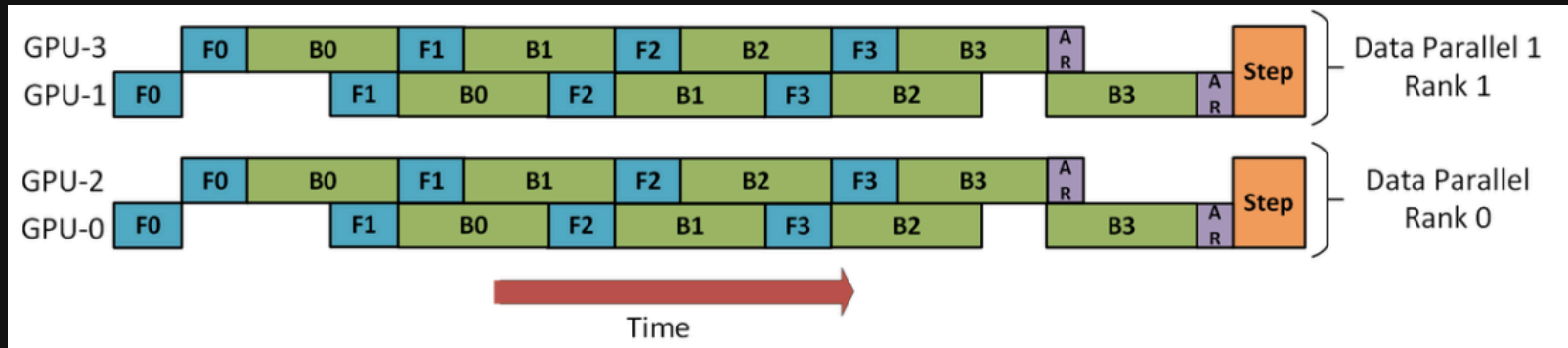
Pipeline Parallelism

pipeline communication



By Hoefler, Torsten, et al. "HammingMesh: A Network Topology for Large-Scale Deep Learning." *Communications of the ACM* 67.12 (2024): 97-105.

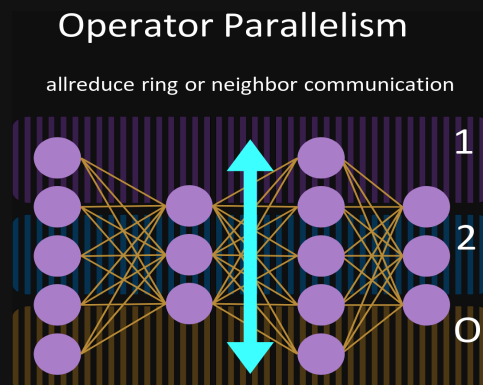
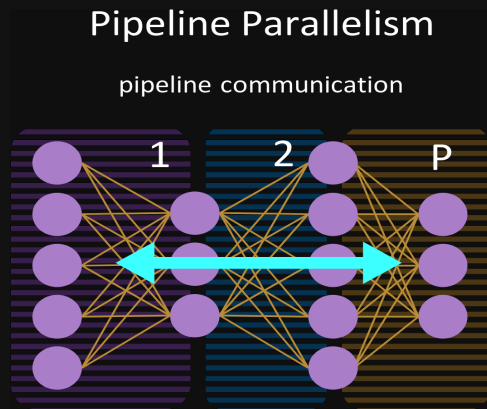
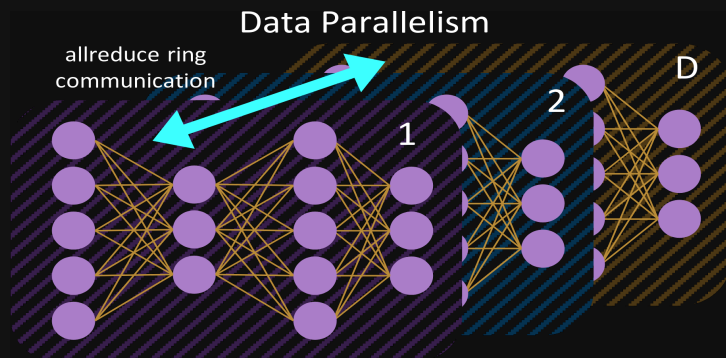
Model Parallelism: Pipeline Parallelism



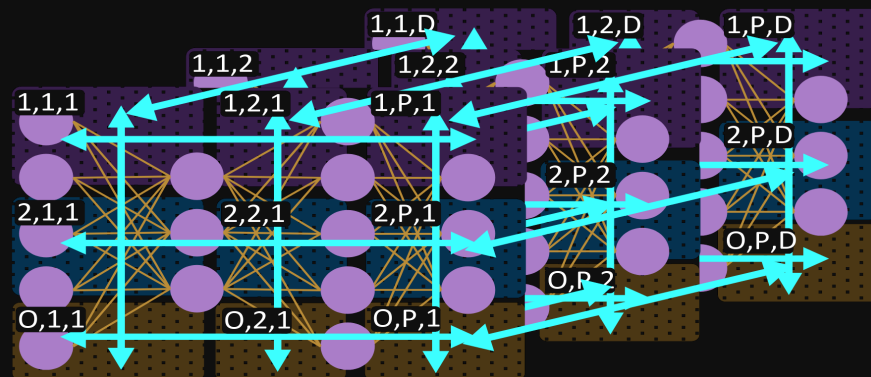
<https://www.deepspeed.ai/tutorials/pipeline/>

- 2-stage pipeline parallelism
- Combined with 2-way data parallelism
- Training of 4 mini-batches

3D Parallelism



3D - Data, Pipeline, and Operator Parallelism



NVIDIA Collective Communication Library (NCCL)

- inter-GPU (collective) communication primitives

Broadcast

Reduce

AllReduce

AllGather

ReduceScatter

Send/Recv

- single kernel implementation for communication and computation
- topology-aware, supports NVLINK
- API is similar to MPI (non-blocking version, e.g. `MPI_Isend`)
- bootstrap for parallel environment is out-of-band (not provided)
- can be used in combination with MPI
- collectives may be faster than GPU-aware MPI
- often used in AI applications (PyTorch, Keras or TensorFlow)
- AMD's version: RCCL

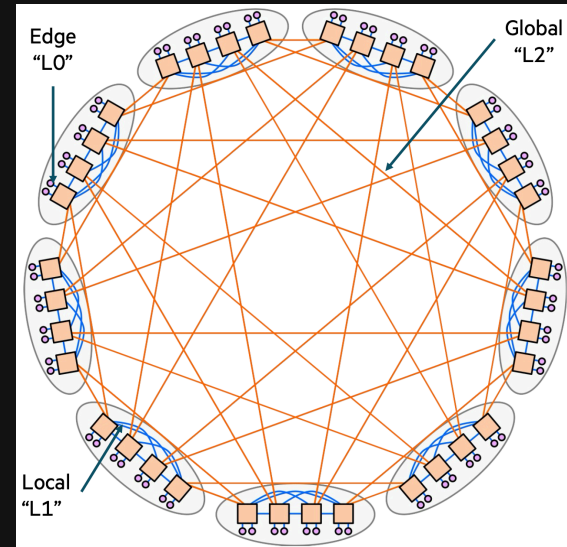
HPE Slingshot Network

- Built from Cassini NIC (PCIe Gen4) and Rosetta Switch (64-port).
 - one NIC (Cassini) per GH200 module (4 per node)
- Uses 200 Gbps Ethernet links for dragonfly networks
 - 25 GB/s unidirectional peak throughput per NIC
- Supports 64 to 250,000+ endpoints (largest system: ~85,000 endpoints).
- Optimized Ethernet protocol
- Adaptive routing & congestion management optimize performance.
- support remote memory operations



Dragonfly Network Topology

- Low-diameter “direct” network with no external fat-tree top switches.
- Fewer optical links needed → reduces cost for large systems.
- Groups of routers act as high-radix virtual routers to improve scalability.
- Three levels of connectivity:
 - Edge links (L0): Nodes to local routers.
 - Intra-group links (L1): Routers within a group.
 - Inter-group links (L2): Optical connections between groups.
- Scales linearly in cost while providing high bandwidth.
- Max three hops between components in the network.



Alps Network Stack

Libfabric

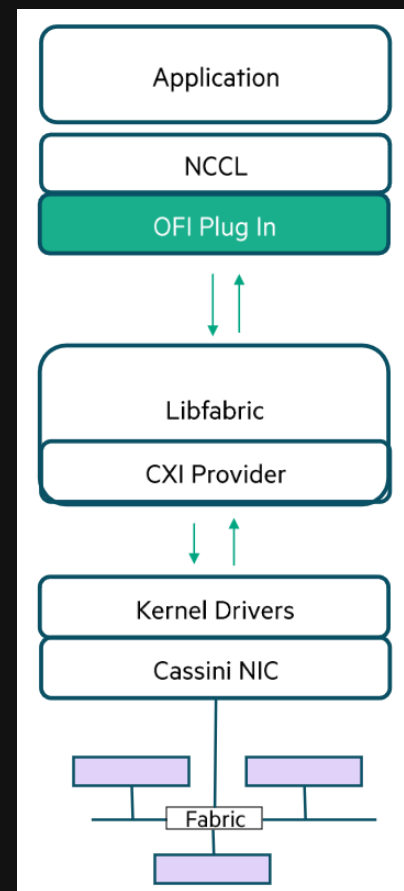
- Open Fabrics Interfaces (OFI)
- low-level communication library
- abstraction over different network technologies
- network specifics are implemented as providers

CXI Provider

- enables libfabric on HPE/Cray's Slingshot network
- makes use of proprietary extensions of the network protocol
- message matching offload to NIC
- RMA and active messages
- GPU-to-NIC RDMA
- Fabric-accelerated small reductions

aws-ofi-plugin

- enables NCCL to use libfabric as network provider
- maps NCCL's connection-oriented transport APIs to libfabric's connection-less reliable interface
 - reliable message support
 - OS bypass
- originally developed by NVIDIA and AWS to support Elastic Fabric Adapter (EFA)
- HPE worked together with NVIDIA and AWS to support Slingshot
- open source: <https://github.com/aws/aws-ofi-nccl>
- fabric and NIC offload capabilities in HPE Slingshot: overlap the compute and communication



torch.distributed

- Core PyTorch package for distributed training
- Initializes communication between processes
- Common backends:
 - `nccl` (recommended for GPU)
 - `gloo` (CPU and fallback)

```
1 import torch.distributed as dist
2
3 dist.init_process_group(
4     backend="nccl",
5     device_id=torch.device(f"cuda:{GPU_ID}"))
6 torch.cuda.set_device(GPU_ID)
```

- By default, PyTorch uses environment variables
 - `RANK` : global rank of the process
 - `LOCAL_RANK` : rank on the current node
 - `WORLD_SIZE` : total number of processes
 - `MASTER_ADDR` : address of the master node
 - `MASTER_PORT` : port for communication

DistributedSampler

- Ensures each process gets a unique slice of the dataset
- Avoids overlapping data between workers
- Should be used with `Dataloader` in distributed training

```
1  from torch.utils.data import DataLoader, DistributedSampler
2
3  # Same arguments as serial DataLoader
4  sampler = DistributedSampler(train_dataset, shuffle=True)
5  # Watch out for the batch size!
6  loader = DataLoader(trainset, batch_size=local_batch_size, sampler=sampler)
```

torch.nn.parallel.DistributedDataParallel (DDP)

- Wraps the model for multi-GPU training
- Each process gets its own model replica

```
1 import torch.nn as nn
2 model = nn.parallel.DistributedDataParallel(model, device_ids=[GPU_ID])
```


torch.distributed.fsdp.FullyShardedDataParallel (FSDP)

- Advanced parallelism: shards parameters, gradients, and optimizer states
- Reduces memory usage significantly
- Useful for large models
- Wraps your model to enable FSDP using

`auto_wrap_policy`

```
1 def custom_auto_wrap_policy(  
2     module, recurse, nonwrapped_numel):  
3     ...
```

```
1 from torch.distributed.fsdp import FullyShardedDataPar  
2 from torch.distributed.fsdp import MixedPrecision, CPU  
3  
4 mixed_precision = MixedPrecision(  
5     param_dtype=torch.float16,  
6     reduce_dtype=torch.float16,  
7     buffer_dtype=torch.float16,  
8 )  
9  
10 model = FSDP(  
11     model,  
12     device_id=device_id,  
13     auto_wrap_policy=custom_auto_wrap_policy,  
14     mixed_precision=mixed_precision,  
15     sharding_strategy=ShardingStrategy.FULL_SHARD,  
16     cpu_offload=CPUOffload(offload_params=False)  
17 )
```

Training Loop Structure

- Loop over epochs
 - Shuffle data using `DistributedSampler.set_epoch(epoch)`
 - Loop over batches (same as serial version!)
 - Forward pass
 - Compute loss
 - Backward pass
 - Optimizer step
- Validation
 - validation loss: synchronization using `dist.all_reduce`

Lab: Distributed Training of CNN

- From a working **single-GPU CNN model** to a **scalable multi-GPU** training
 - **Distributed Data Parallel (DDP)**
 - **Fully Sharded Data Parallel (FSDP)**
- In the end: Strong scaling study (fixed **global batch size**)
 - How does the training time change with the number of GPUs
- Slurm job submission (no JupyterLab)

SSH Access to CSCS Cluster

- For a convenience, it is recommended to add the following lines to your

`~/.ssh/config` file

```
1 Host ela
2     HostName ela.cscs.ch
3     User <YOUR_USERNAME>
4     ForwardAgent yes
5     ForwardX11 yes
6
7 Host daint
8     HostName daint.cscs.ch
9     User <YOUR_USERNAME>
10    ProxyJump ela
11    ForwardAgent yes
12    ForwardX11 yes
```

- After that, you can connect to the cluster using the command below

```
1 $ ssh <YOUR_USERNAME>@daint
```

- You should be dropped into the `bash` shell in your home directory
- For launching jobs and we use the scratch directory

```
1 $ cd $SCRATCH/pytorch-training/
```

- pull the latest changes

```
1 $ git pull
2 $ cd notebooks/distributed
```



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

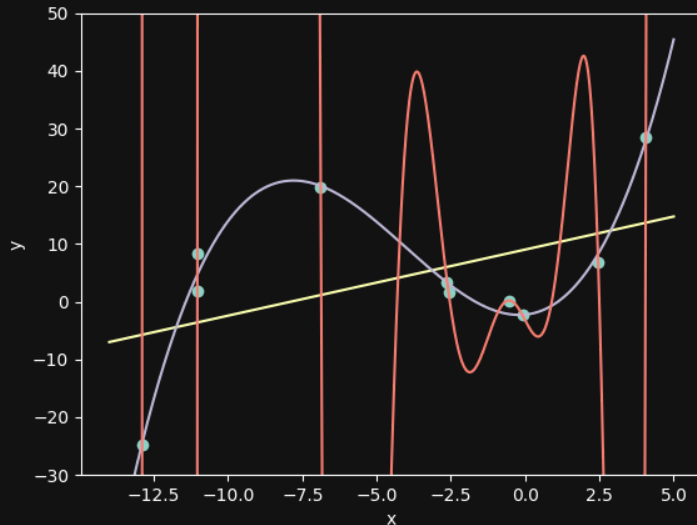
ETH zürich

More on Training

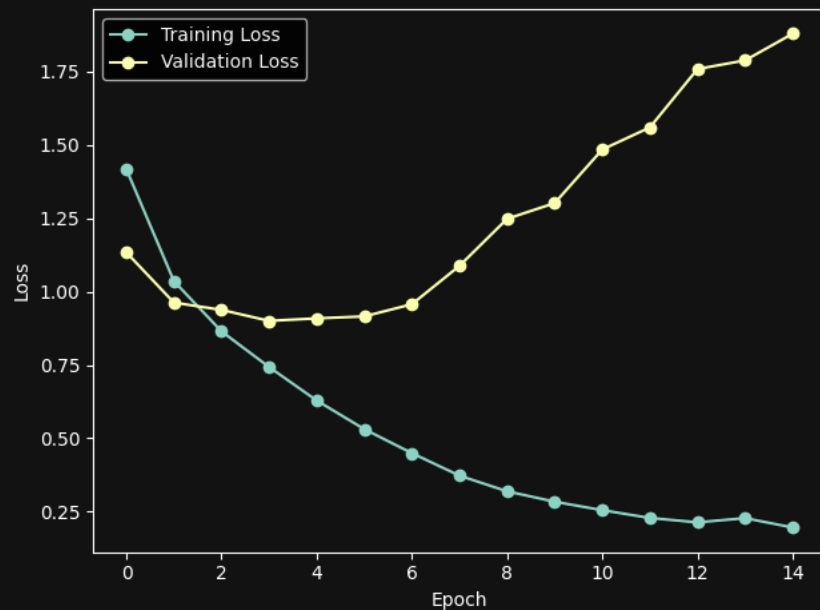
Overfitting

Overfitting happens when a model has too many parameters compared to the training data.

$$f(x) \approx \sum_i w_i x^i$$



Overfitting and Learning Curves



Low training loss and high validation loss indicate overfitting.

The large gap represents the generalization error.

The model was performing better early on in the training, but then started to overfit.

Early Stopping

Stop training when the validation error reaches a minimum.

A trivial implementation of early stopping is to save the best model.

PyTorch: State Dictionary `state_dict`

In PyTorch, learnable parameters (weights and biases) are stored in `torch.nn.Model` 's parameters.

```
1 model.parameters()
```

A *state dictionary* maps each layer of the model to each parameter.

```
1 model.state_dict()
```

Optimizers (`torch.optim`) also have a state dictionary, which is useful to resume training.

PyTorch: Save and Load Parameters to/from File

Saving

```
1 model = ...
2 optimizer = ...
3
4 torch.save(
5     model.state_dict(), PATH
6 )
7 torch.save(
8     optimizer.state_dict(), PATH
9 )
```

Loading

```
1 model = ...
2 optimizer = ...
3
4 model.load_state_dict(
5     torch.load(PATH)
6 )
7 optimizer.load_state_dict(
8     torch.load(PATH)
9 )
```

PyTorch: Save and Load Parameters

`state_dict()` returns a reference to the model's parameters.

Warning

Use a deep copy to copy the state dictionary.

```
1  from copy import deepcopy
2
3  model = ...
4  best_model_state = deepcopy(model.state_dict())
```

Regularization: Loss Function

We can add constraints on the model (model weights) via the loss function.

Regularization puts constraints on the model weights:

- L_2 regularization encourages decay towards zero unless supported by data
- L_1 regularization encourages sparsity (many weights close to zero)

Controlling the model complexity by regularization has similar effects as reducing the model capacity (number of parameters).

PyTorch: L_2 Regularization

$$L_2 = \lambda \sum_i w_i^2$$

The L_2 regularization term is available in PyTorch via the `weight_decay` argument of the optimizer.

```
1 import torch.optim as optim
2
3 optimizer = optim.SGD(
4     model.parameters(),
5     lr=0.001,
6     weight_decay=0.01
7 )
```

PyTorch: L_1 Regularization

$$L_1 = \lambda \sum_i |w_i|$$

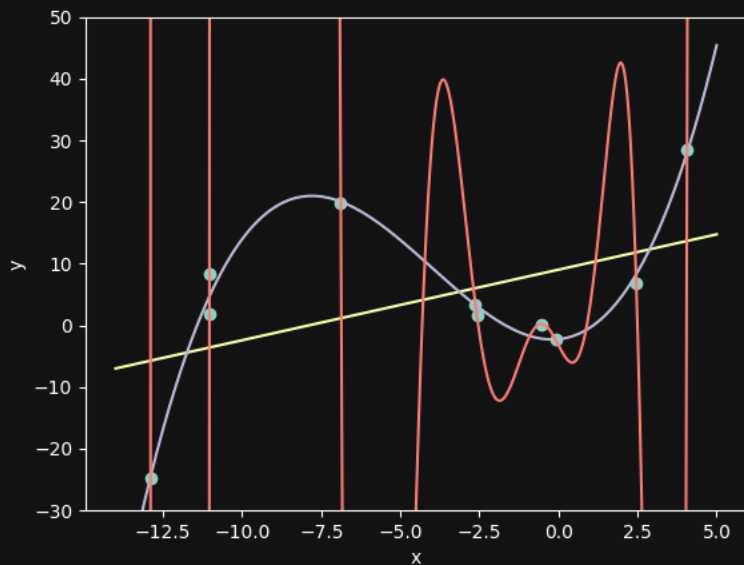
The L_1 regularization term produces a sparse model (many weights close to zero).

This term is not available in PyTorch optimizers, but can be implemented manually.

```
1  def l1_loss(params, lambda):
2      l1 = 0
3      for p in params:
4          l1 += torch.sum(torch.abs(p))
5
6      return lambda * l1
7
8  total_loss = loss + l1_loss(model.parameters(), lambda = 0.01)
9  total_loss.backward()
```

Regularization: Reduce Model Capacity

Reducing the model capacity (number of parameters) can reduce over-fitting.



Number of Parameters

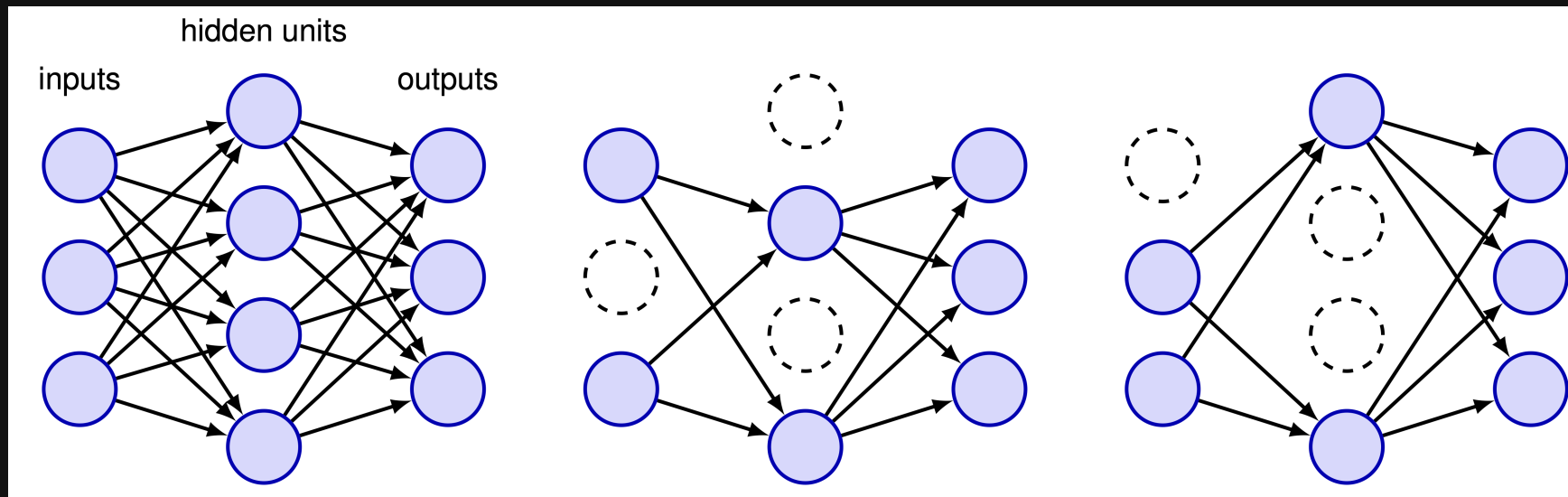
```
1 num_elements_list = [  
2     p.numel()  
3     for p in model.parameters()  
4     if p.requires_grad # Include trainable parameters  
5 ]  
6  
7 sum(num_elements_list)
```

Regularization: Dropout Layers

Dropout layers randomly zero some elements of the input with probability p .

The output is scaled by $1/(1 - p)$.

```
1 nn.Dropout(p=0.5)
```



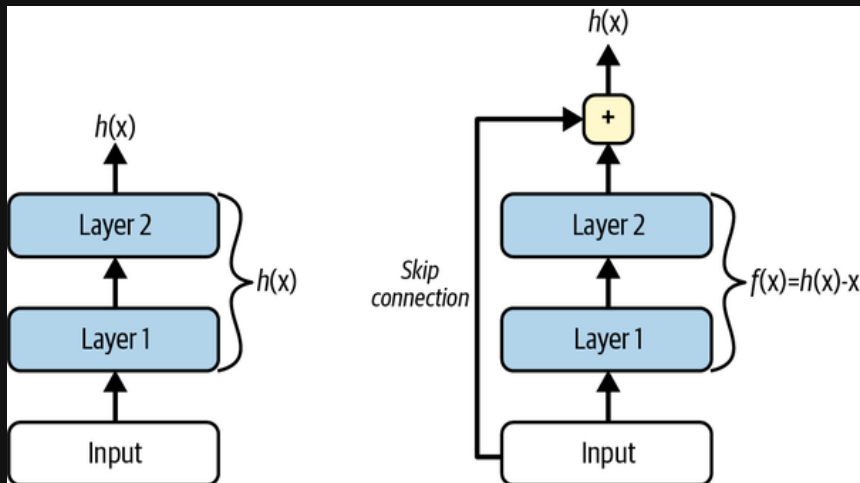
Training and Evaluation Phases

Some layers behave differently during training and evaluation (dropout, batch normalization, ...).

```
1 model.train()
```

```
1 model.eval()
```

Skip Connections and Residual Learning



- Speed-up training
- Facilitate signal propagation
- Reduce vanishing/exploding gradients problem

```
1 # torchvision/models/resnet.py
2 def forward(self, x):
3     identity = x
4
5     out = self.conv1(x)
6     out = self.bn1(out)
7     out = self.relu(out)
8
9     out = self.conv2(out)
10    out = self.bn2(out)
11
12    out += identity
13    out = self.relu(out)
14
15    return out
```

Here we can't use `nn.Sequential`.

Géron, Aurélien. Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow, O'Reilly Media, Inc., 2022

Data Augmentation

Data augmentation artificially inflates the size of the training set by transforming the data (on-the-fly) at each iteration.

- Random rotations
- Random translations
- Random flips
- Random crops
- ...

Data augmentation helps reduce overfitting, and improves generalization.

PyTorch: Data Augmentation

```
1 import torch
2 from torchvision.transforms import v2
3
4 transforms = v2.Compose([
5     v2.RandomResizedCrop(size=(224, 224)),
6     v2.RandomHorizontalFlip(p=0.5),
7     v2.ToDtype(torch.float32, scale=True),
8     v2.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
9 ])
```

Optimizers

SDG takes small, regular steps:

$$\theta_t = \theta_{t-1} - \eta \nabla_{\theta_{t-1}} \mathcal{L}(\theta_{t-1})$$

Momentum optimization adds inertia to the motion through weight space:

$$\mathbf{m}_t = -\eta \nabla_{\theta_{t-1}} \mathcal{L}(\theta_{t-1}) - \beta \mathbf{m}_{t-1}$$

$$\theta_t = \theta_{t-1} + \mathbf{m}_t$$

Adaptive Learning Rate

A fixed learning (`lr`) rate might not be optimal for the whole training.

- Adjust `lr` based on the number of epochs
- Adjust `lr` based on validation metrics

```
1  from torch.optim.lr_scheduler import ExponentialLR
2
3  optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
4  scheduler = ExponentialLR(optimizer, gamma=0.9)
5
6  for epoch in range(20):
7      for input, target in dataset:
8          optimizer.zero_grad()
9          output = model(input)
10         loss = loss_fn(output, target)
11         loss.backward()
12         optimizer.step()
13     scheduler.step()
```

Weight Initialization

- Weights are randomly initialized
 - Uniform, normal, Xavier, ... distributions
- Initialization has an impact on training
- PyTorch uses sensible choices



<https://xkcd.com/1838/>

PyTorch: Weight Initialization

Weight initialization can be customized.

```
1  def custom_weights_init(module):
2      if isinstance(module, nn.Linear):
3          module.weight.data.normal_(mean=0.0, std=1.0)
4          if module.bias is not None:
5              module.bias.data.zero_()
6
7      if isinstance(module, nn.Conv2D):
8          # Use torch.nn.init module
9          torch.nn.init.xavier_uniform_(module.weight.data)
10
11  model.apply(custom_weights_init)
```


Transfer Learning and Fine-Tuning: Motivation

- Training a deep learning model can be very expensive
- Many tasks are closely related
- Some data sets are too small to train a performant deep learning model
- Random initialization is far from optimal

Transfer learning and fine-tuning aim at re-using pre-trained models and fine tune them for the specific task at hand.

Transfer Learning and Fine-Tuning: CNN example

CNNs for classification have two conceptual building blocks:

- Feature extractor (convolution layers)
- Classifier (linear layers)

Feature extraction layers can be re-used from successfully trained models, while the classifier can be re-trained for the task at hand.

PyTorch: Load torchvision Pre-Trained Models

```
1 from torchvision.models import resnet50, ResNet50_Weights
2
3 resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
4 resnet50(weights=ResNet50_Weights.DEFAULT)
5 resnet50(weights="IMAGENET1K_V2")
6
7 # No weights (random initialization)
8 resnet50(weights=None)
```

PyTorch: Freeze Model Parameters

To freeze model parameters, do not compute gradients during the backward pass:

```
1  for param in model_conv.parameters():  
2      param.requires_grad = False
```

PyTorch: Overwrite Pre-Trained Layers

```
1 model = nn.Sequential(  
2     nn.Linear(784, 256),  
3     nn.ReLU(),  
4     nn.Linear(256, 64),  
5     nn.ReLU(),  
6     nn.Linear(64, 10)  
7 )  
8  
9 print(model)  
10 # Sequential(  
11 #   (0): Linear(...)  
12 #   (1): ReLU()  
13 #   (2): Linear(...)  
14 #   (3): ReLU()  
15 #   (4): Linear(...)  
16 # )
```

```
1 for param in model.parameters()[::-1]:  
2     param.requires_grad = False  
3  
4 n_in_features = model[-1].in_features  
5  
6 # Swap last linear layer  
7 # with another (untrained) linear layer  
8 model[-1] = nn.Linear(n_in_features, 5)
```

PyTorch Ecosystem

Hyperparameters can have a huge impact on the model, and the hyperparameter space is large.

- Ray Tune (<https://docs.ray.io>)
- Optuna (<https://optuna.org>)
- Hyperopt (<http://hyperopt.github.io/hyperopt/>)
- ...

PyTorch is rather bare-bones. There are many libraries built on top of it which require to write less code:

- PyTorch Ignite (<https://pytorch-ignite.ai>)
- PyTorch Lightning (<https://lightning.ai/pytorch-lightning>)
- FastAI (<https://www.fast.ai>)
- Keras 3.0 (<https://keras.io>)
- ...

Keras 3.0 Example

Model Definition

```
1 model = keras.Sequential([
2     keras.Input(shape=input_shape),
3     layers.Conv2D(32,
4         kernel_size=(3, 3),
5         activation="relu"
6     ),
7     layers.MaxPooling2D(
8         pool_size=(2, 2)
9     ),
10    ...
11    layers.Flatten(),
12    layers.Dense(
13        num_classes,
14        activation="softmax"
15    )])
```

Training and Inference

```
1 model.compile(
2     loss="categorical_crossentropy",
3     optimizer="adam",
4     metrics=["accuracy"]
5 )
6
7 model.fit(
8     x_train, y_train,
9     batch_size=batch_size,
10    epochs=epochs,
11    validation_split=0.1
12 )
13
14 score = model.evaluate(
15     x_test, y_test, verbose=0
16 )
```

[LAB] Regularization, Data Augmentation, and Transfer Learning

Regularization and data augmentation:

- Train a CNN with and without dropout layers
- Train a CNN with and without data augmentation
- Save and load model weights (early stopping)

Transfer learning (optional):

- Modify VGG-19 to work with only 5 classes
- Freeze retained VGG-19 parameters
- Apply transfer learning to the new data